CV802

Assignment 2: Surface Extraction

Anas Khan

Problem Statement:



points | Poisson | IER | Point2Mesh | MeshAnything | DHSP | NKSR | SAP

VGGSfM points from Assignment 1 → textured meshes (.obj)

Inputs


VGGSfM front
138,677 pts
VGGSfM back
MVS front
487,451 pts
MVS back
MVS-clean front
487,047 pts
MVS-clean back

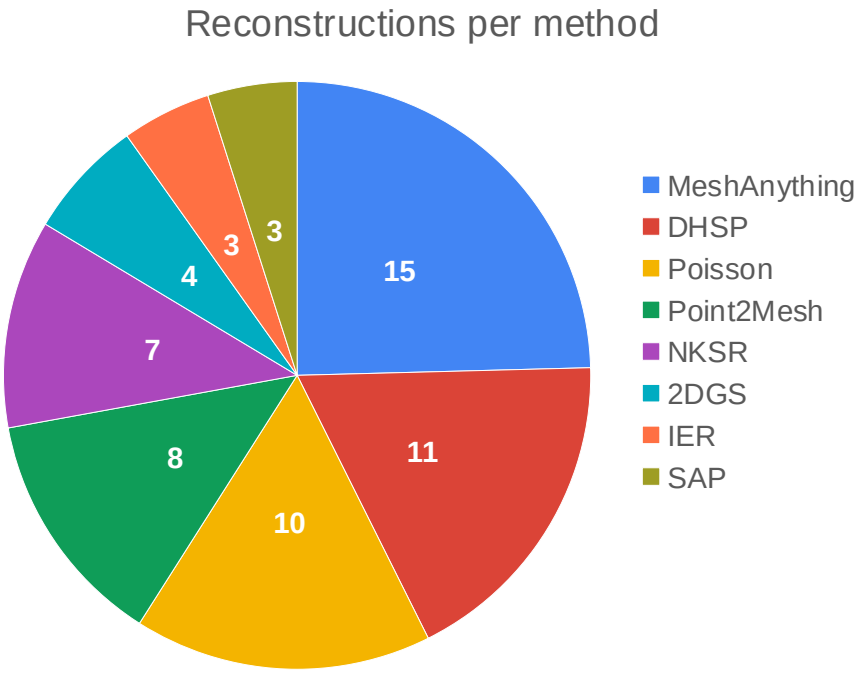
Methods

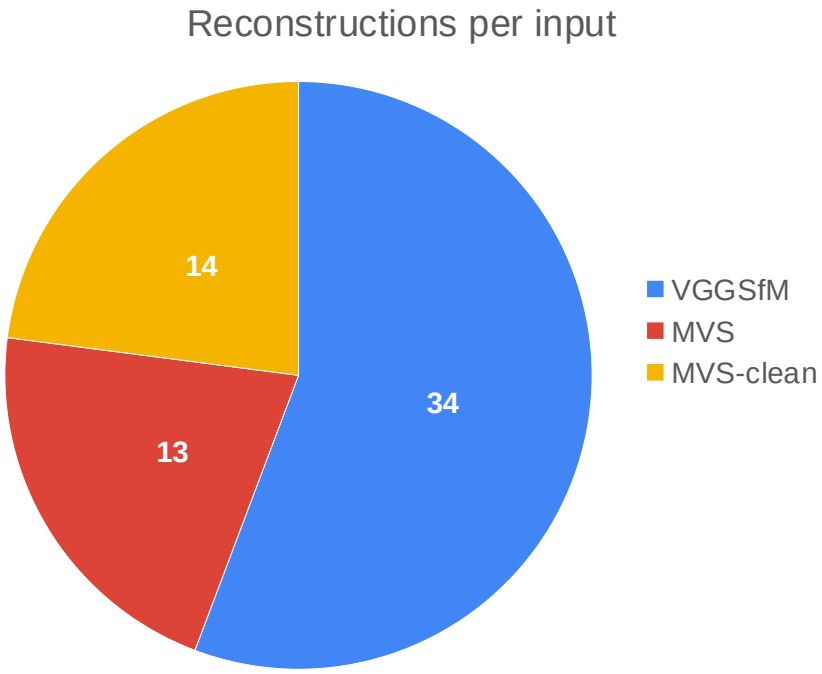
Method	Venue	Year
Screened Poisson (our implementation)	SGP / ACM TOG	2006 / 2013
IER meshing	ECCV	2020
Point2Mesh	SIGGRAPH	2020
MeshAnything V2	arXiv	2024
Deep Hybrid Self-Prior (DHSP)	ICCV	2021
NKSR*	CVPR	2023
Shape As Points (SAP)*	NeurIPS	2021
2D Gaussian Splatting (2DGS)*	SIGGRAPH	2024

* beyond the course list

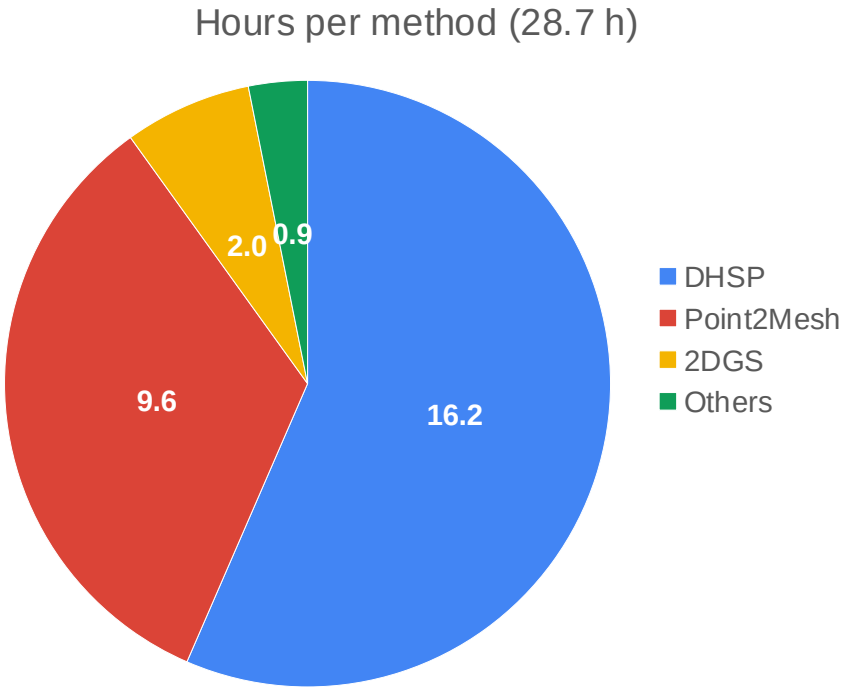
Experiments

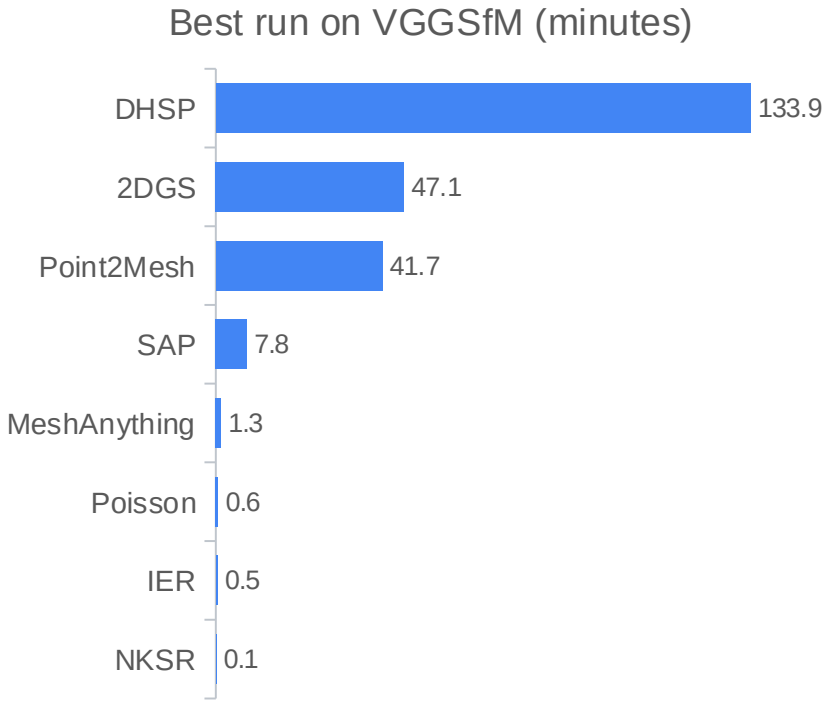
61 reconstructions in total





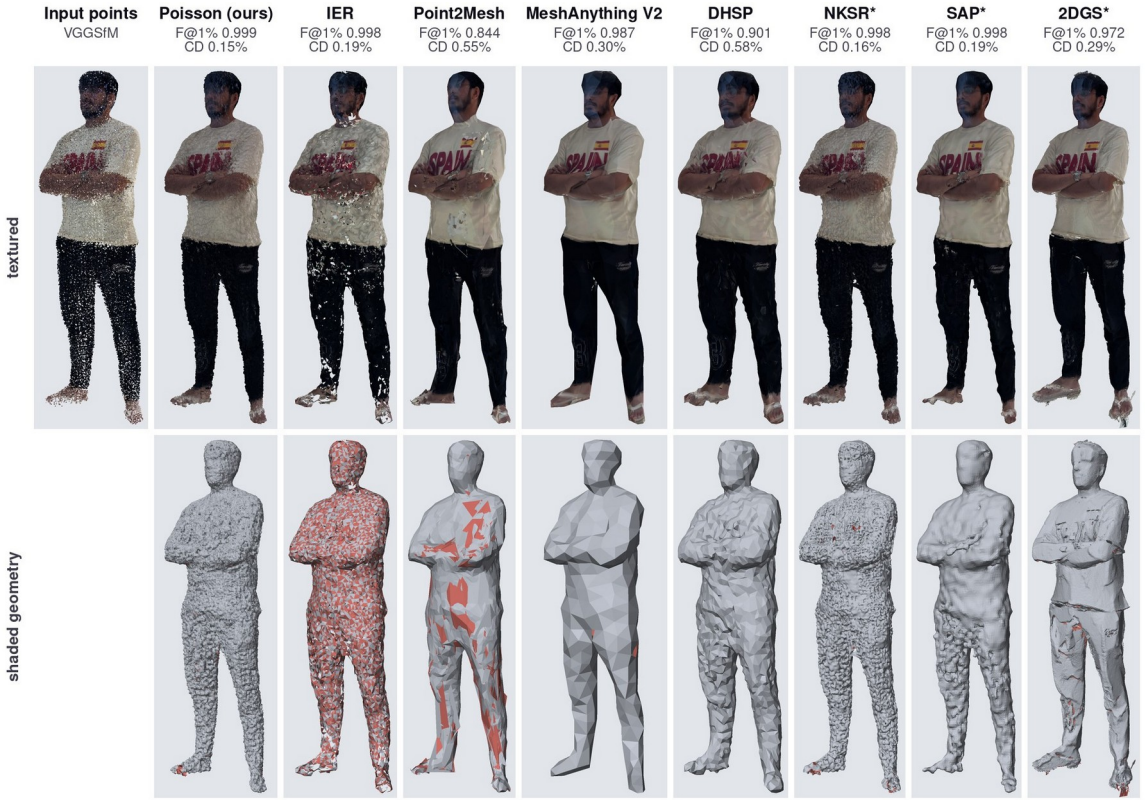
GPU time





One RTX 5000 Ada (32 GB). Others: SAP, MeshAnything, Poisson, IER, NKSR. DHSP includes its 3D and 2D stages. Not counted: failed attempts, e.g. a 15 h flattening run that never finished.

Results (VGGSfM)



Comparison (½)

best second best

Course list

Method	Faces	Chamfer % ↓	F@1% ↑	F@0.5% ↑	NC ↑	Watertight	Time ↓
Poisson (ours)	**1.02M**	**0.149**	**0.999**	**0.988**	**0.75**	**no**	**39 s**
IER	**17k**	**0.187**	**0.998**	**0.975**	**0.82**	**no**	**29 s**
Point2Mesh	**10k**	**0.553**	**0.844**	**0.530**	**0.80**	**yes**	**42 min**
MeshAnything V2	**1.4k**	**0.295**	**0.987**	**0.858**	**0.82**	**no**	**77 s**
DHSP	**6.0k**	**0.583**	**0.901**	**0.407**	**0.89**	**yes**	**2.2 h**

Beyond the course list

Method	Faces	Chamfer % ↓	F@1% ↑	F@0.5% ↑	NC ↑	Watertight	Time ↓
NKSR*	**2.09M**	**0.156**	**0.998**	**0.973**	**0.75**	**no**	**7 s**
SAP*	**64k**	**0.195**	**0.998**	**0.961**	**0.81**	**yes**	**8 min**
2DGS*	**657k**	**0.289**	**0.972**	**0.864**	**0.75**	**no**	**47 min**

VGGSfM input (138,677 points), best run per method.

Comparison (2/2)

best second best

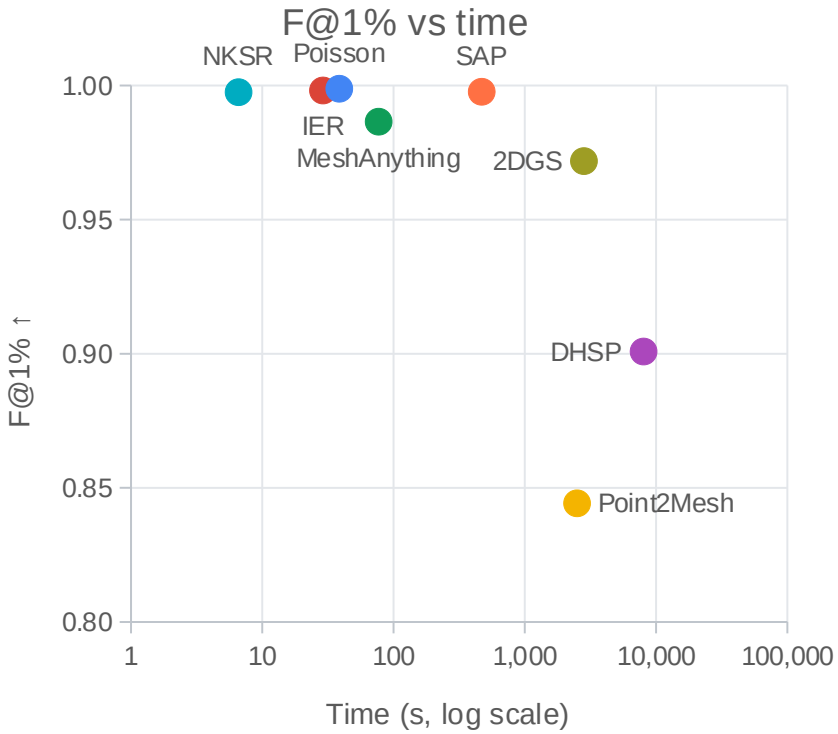
Course list

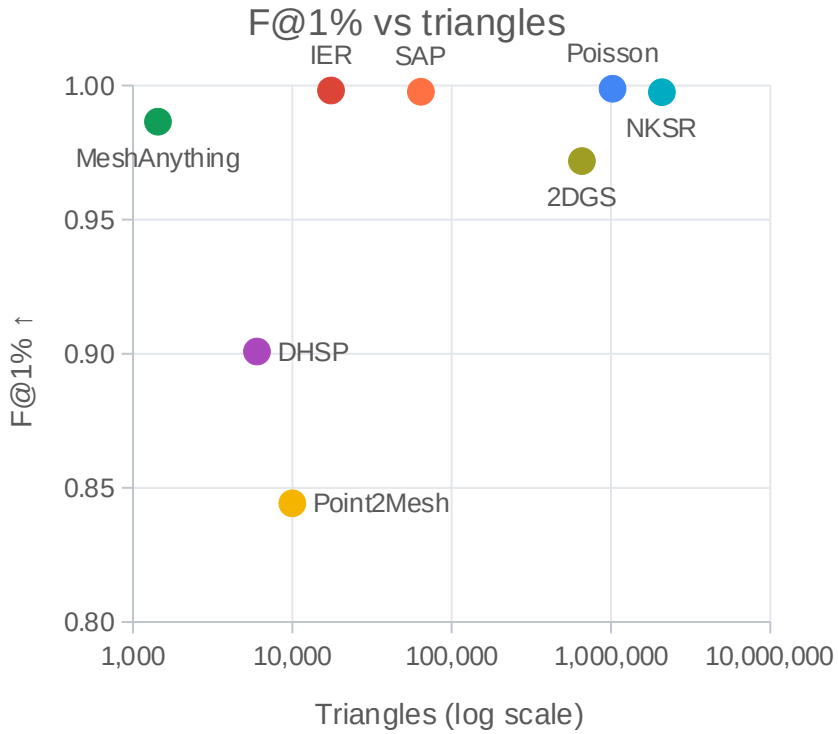
Method	MVS (487,451 points)			MVS-clean (487,047 points)		
	F@1% ↑	Chamfer % ↓	Time ↓	F@1% ↑	Chamfer % ↓	Time ↓
Poisson (ours)	**0.999**	**0.127**	**41 s**	**0.999**	**0.127**	**40 s**
IER	**1.000**	**0.120**	**42 s**	**1.000**	**0.123**	**50 s**
Point2Mesh	**0.133**	**3.977**	**40 min**	**0.387**	**1.298**	**39 min**
MeshAnything V2	**0.873**	**0.518**	**2 min**	**0.908**	**0.438**	**41 s**
DHSP	**0.900**	**0.636**	**4.0 h**	**0.916**	**0.627**	**5.9 h**

Beyond the course list

Method	MVS (487,451 points)			MVS-clean (487,047 points)		
	F@1% ↑	Chamfer % ↓	Time ↓	F@1% ↑	Chamfer % ↓	Time ↓
NKSR*	**1.000**	**0.080**	**12 s**	**1.000**	**0.080**	**18 s**
SAP*	**0.963**	**0.197**	**9 min**	**0.948**	**0.239**	**5 min**
2DGS*	**0.938**	**0.308**	**30 min**	**0.928**	**0.336**	**16 min**

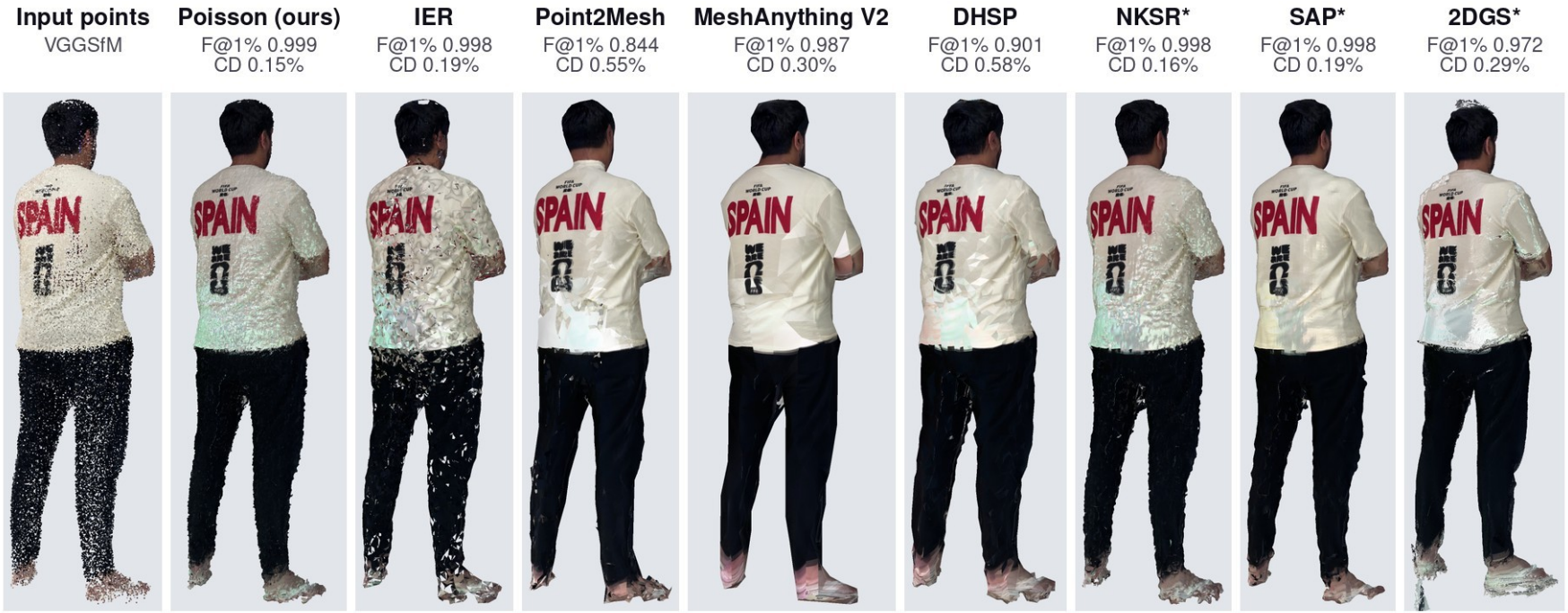
Quality vs time and size



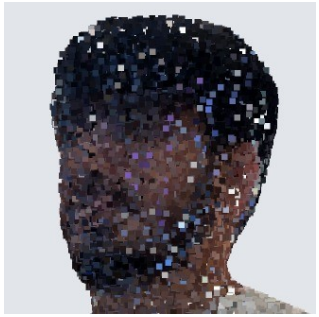


VGGSfM input, best run per method. Higher is better; further left is faster (left chart) or lighter (right chart).

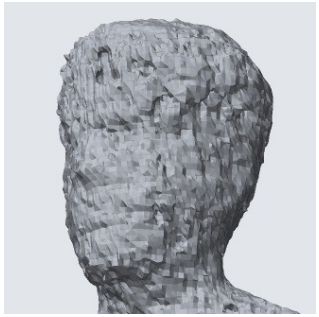
Back view (VGGSfM)



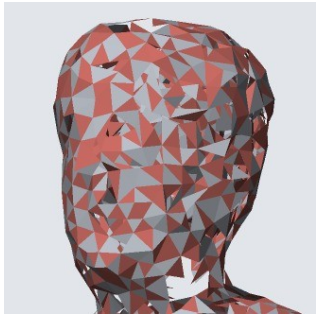
Close-up: face



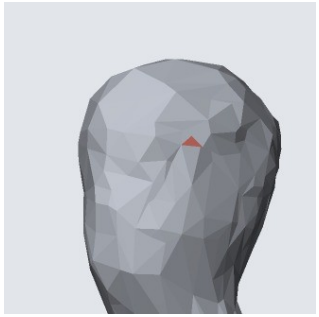
Input points



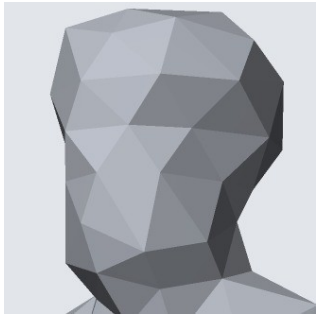
Poisson (ours)



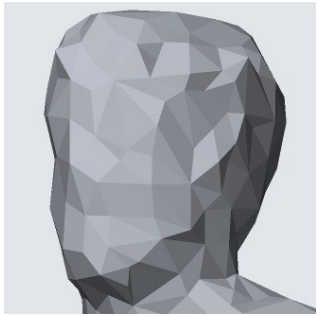
IER



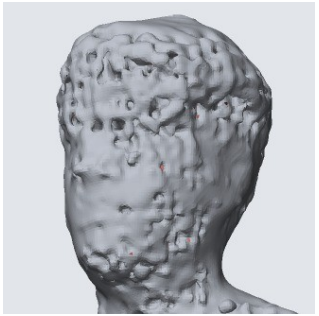
Point2Mesh



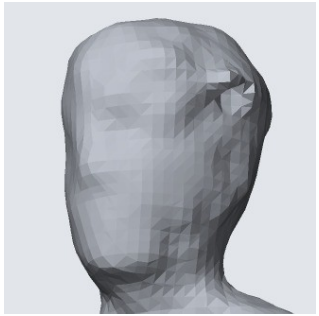
MeshAnything V2



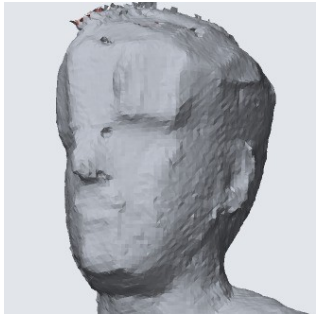
DHSP



NKSR*



SAP*



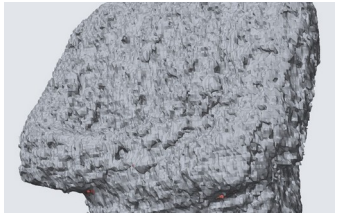
2DGS*

Shaded geometry of the best VGGSfM run per method, same crop for all. Red: triangles seen from the back.

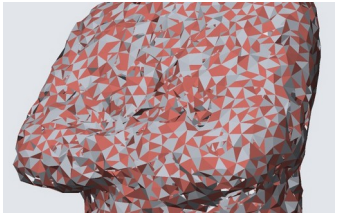
Close-up: crossed arms



Input points



Poisson (ours)



IER



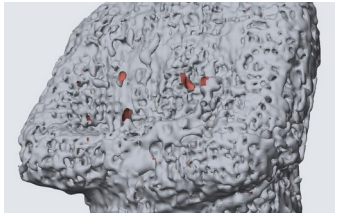
Point2Mesh



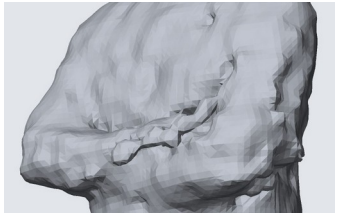
MeshAnything V2



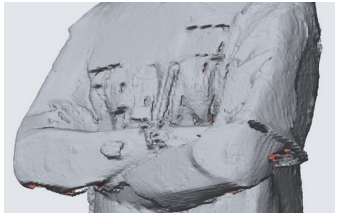
DHSP



NKSR*



SAP*



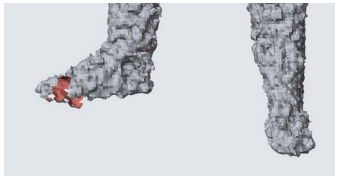
2DGS*

Shaded geometry of the best VGGSfM run per method, same crop for all. Red: triangles seen from the back.

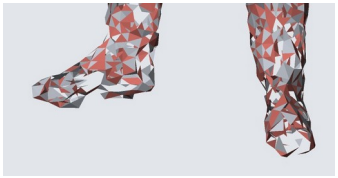
Close-up: feet



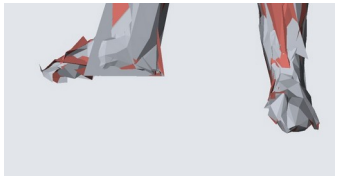
Input points



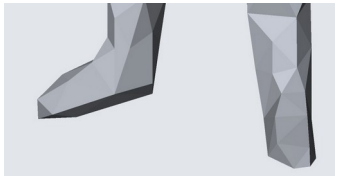
Poisson (ours)



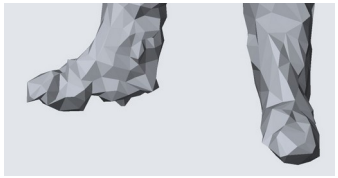
IER



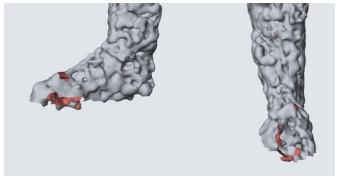
Point2Mesh



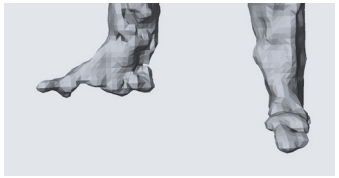
MeshAnything V2



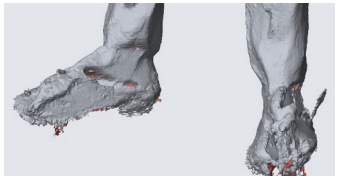
DHSP



NKSR*



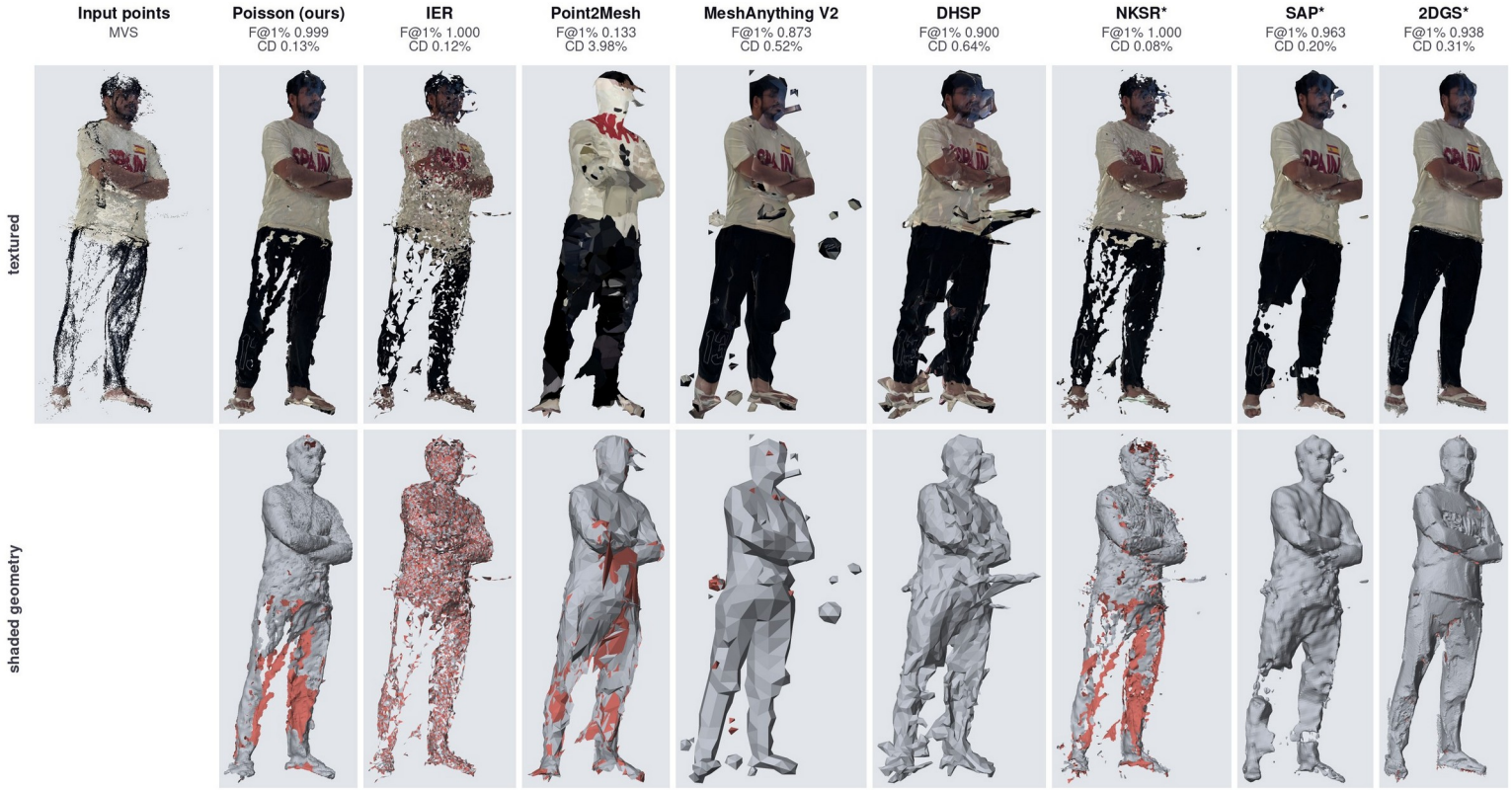
SAP*



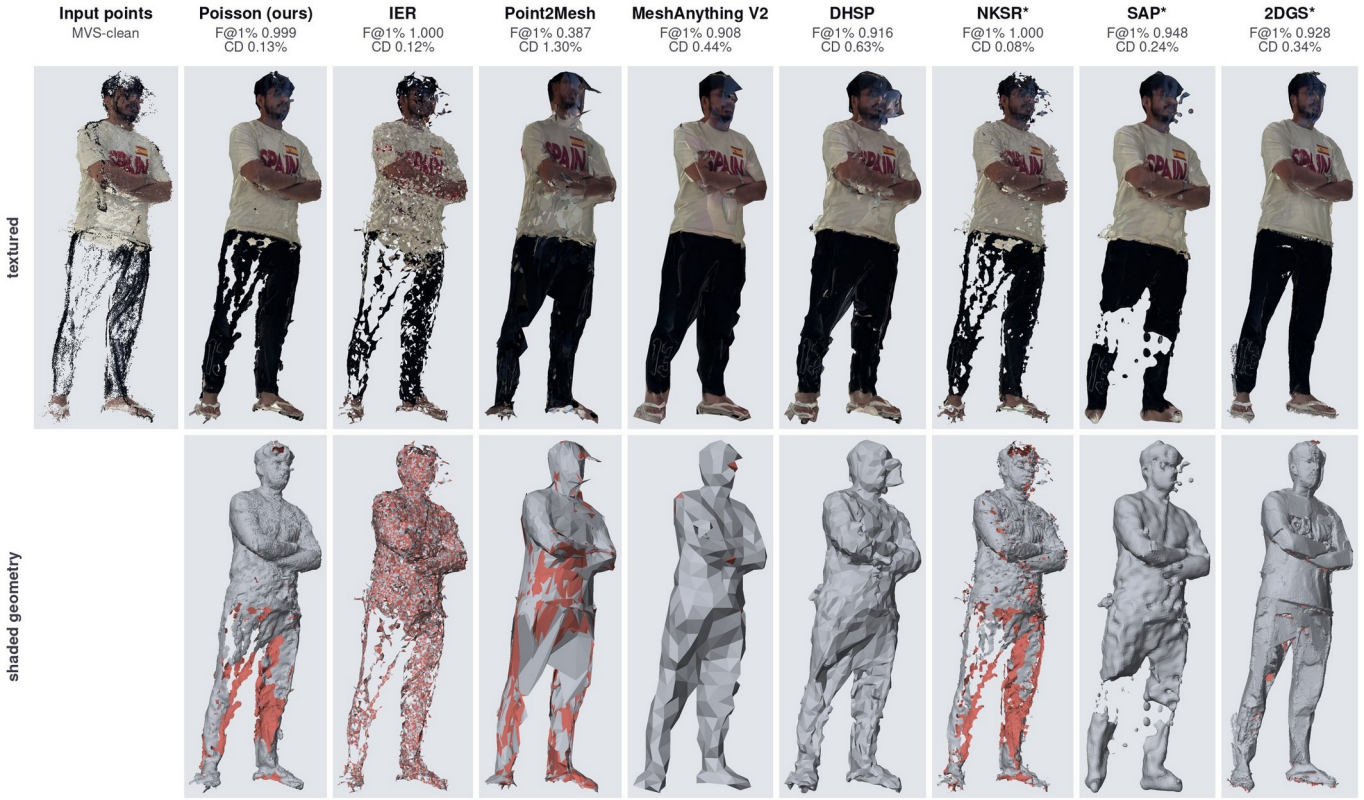
2DGS*

Shaded geometry of the best VGGSfM run per method, same crop for all. Red: triangles seen from the back.

Results (MVS)


Input points
MVS
Poisson (ours)
F@1% 0.999
CD 0.13%
IER
F@1% 1.000
CD 0.12%
Point2Mesh
F@1% 0.133
CD 3.98%
MeshAnything V2
F@1% 0.873
CD 0.52%
DHSP
F@1% 0.900
CD 0.64%
NKSR*
F@1% 1.000
CD 0.08%
SAP*
F@1% 0.963
CD 0.20%
2DGS*
F@1% 0.938
CD 0.31%
textured
shaded geometry

Results (MVS-clean)



Poisson experiments

best second best

256^3, no screening
Chamfer 0.230%



256^3, screened
Chamfer 0.179%



384^3, screened
Chamfer 0.149%



384^3, not trimmed
Chamfer 0.154%



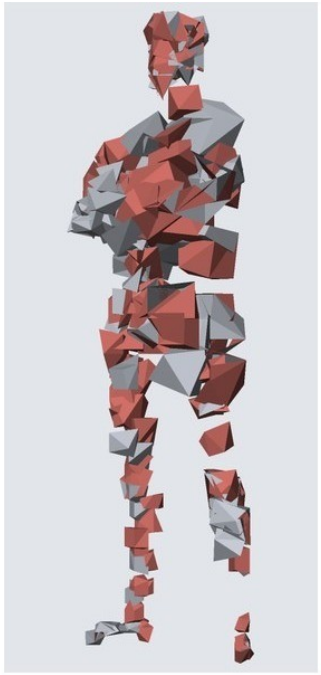
Run	Faces	F@1%	Time
256^3, no screening	**265k**	**0.990**	**14 s**
256^3, screened	**358k**	**0.998**	**17 s**
384^3, screened	**1.02M**	**0.999**	**39 s**
384^3, not trimmed	**1.05M**	**0.998**	**25 s**

Screened: surface pulled onto the points
No screening: follows point directions only
Trimmed: parts far from points are cut off

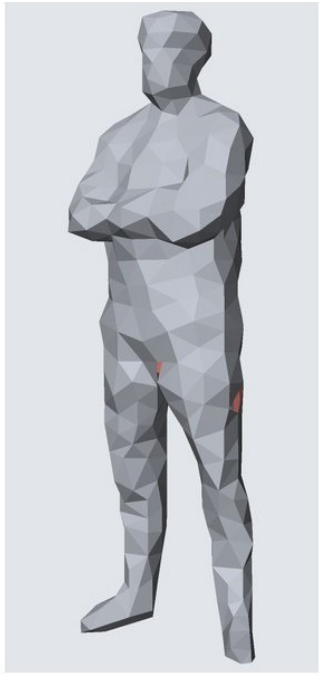
MeshAnything experiments

best second best

raw points
F@1% 0.508 · 1,802 faces



Poisson mesh as input
F@1% 0.987 · 1,435 faces



Input	Model	Decoding	Faces	F@1%	Time
raw points	V2	greedy	624	0.113	92 s
raw points	V2	sampling	1,802	0.508	77 s
raw points	V1	greedy	795	0.442	59 s
Poisson mesh	V1	greedy	471	0.929	51 s
Poisson mesh	V2	sampling	2,058	0.965	95 s
Poisson mesh	V2	greedy	1,435	0.987	77 s

Point2Mesh experiments

best second best

paper defaults
F@1% 0.646



unoriented
F@1% 0.844



MVS input
F@1% 0.133



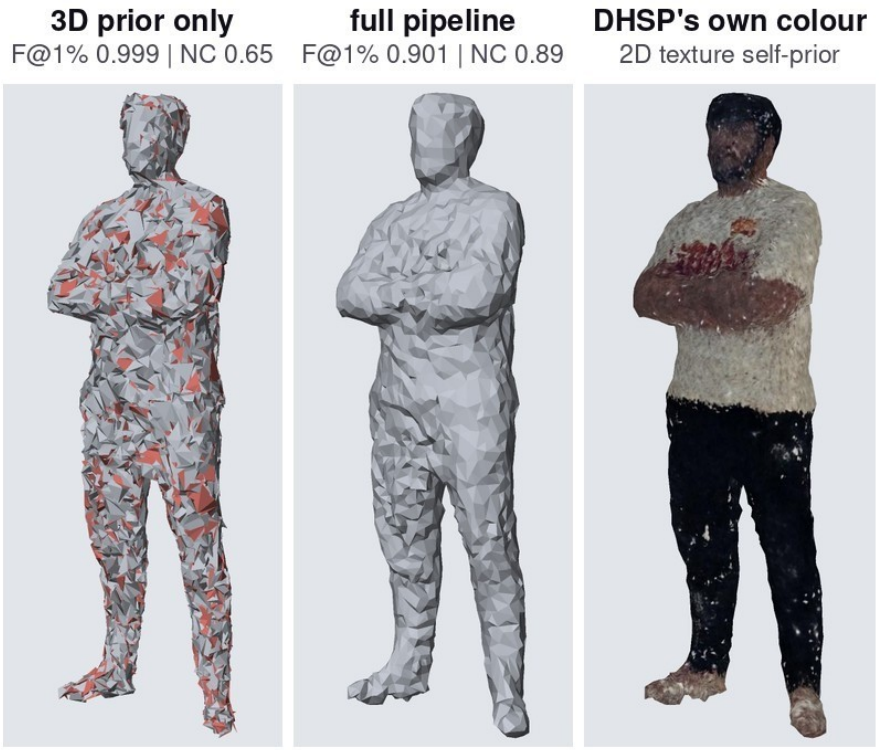
MVS-clean
F@1% 0.387



Run	Faces	F@1%	Time
first try	16k	0.602	21 min
paper defaults	10k	0.646	44 min
+ unoriented	10k	0.844	42 min
+ unoriented, 20k	20k	0.655	3.4 h
MVS	10k	0.133	40 min
MVS-clean	10k	0.387	39 min

DHSP experiments

best second best



Run	Faces	F@1%	NC	Time
3D prior only	**16k**	**0.999**	**0.65**	**1.8 h**
full pipeline	**6k**	**0.874**	**0.89**	**4.1 h**
full, extra smoothing	**6k**	**0.901**	**0.89**	**2.2 h**
MVS, full	**6k**	**0.900**	**0.69**	**4.0 h**
MVS-clean, full	**6k**	**0.916**	**0.72**	**5.9 h**

Full pipeline time = 3D stage + 2D stage.

IER experiments

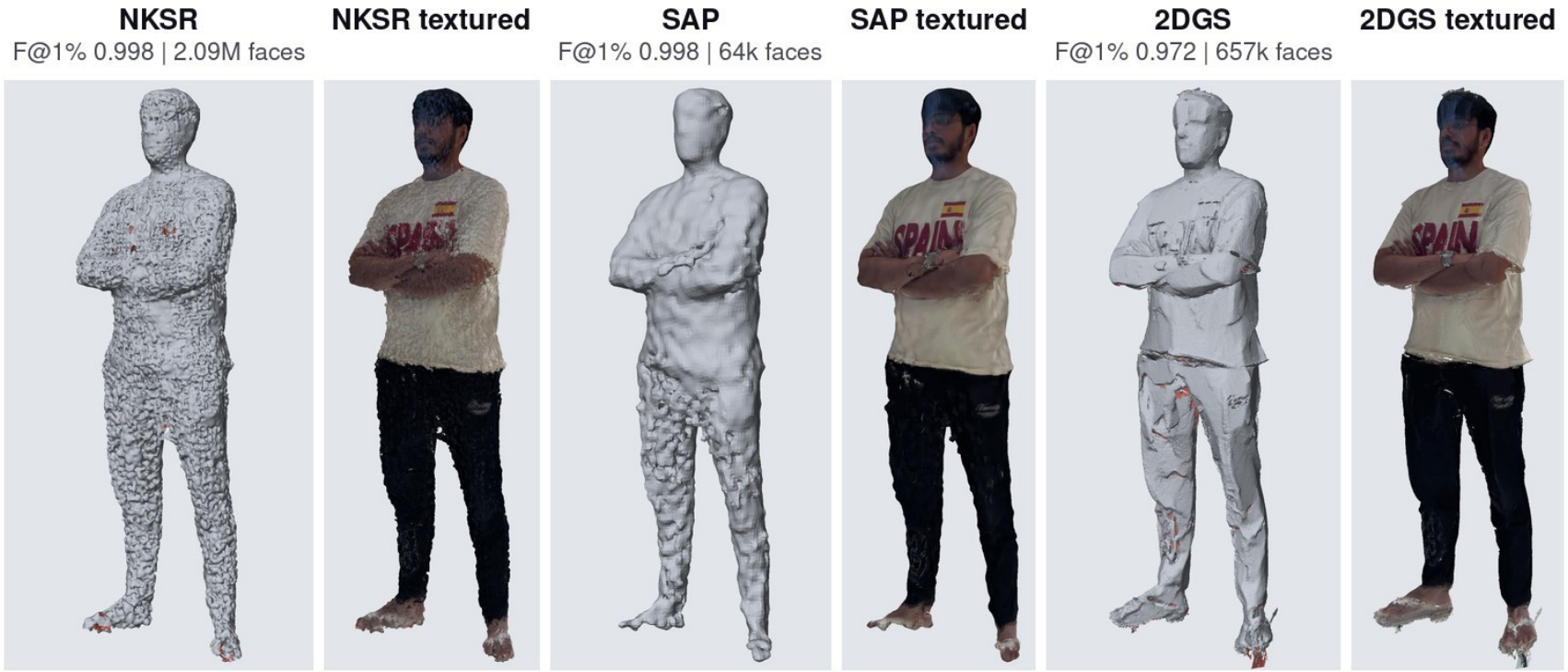
best second best





Input	Faces	F@1%	Time
VGGSfM	17.5k	0.998	29 s
MVS	16.4k	1.000	42 s
MVS-clean	16.7k	1.000	50 s

NKSR, SAP and 2DGS



NKSR: 2.09M faces in 7 s SAP: 64k faces in 8 min 2DGS: 657k faces in 47 min

Findings

1. Our own Poisson ties the best learned methods on VGGSfM (F@1% 0.999).
2. NKSR is best on the dense MVS clouds (Chamfer 0.08%).
3. A high score can hide a crumpled surface (DHSP 3D prior: F@1% 0.999, NC 0.65).
4. The input matters: MeshAnything 0.51 → 0.987 with a mesh input;
   Point2Mesh 0.13 → 0.39 after removing MVS floaters.
5. Four methods give closed (watertight) meshes.

Let’s go through the videos!

Disclaimer

The next seven videos took a lot of effort to make.

Every scene was planned, animated and checked against our own runs, so every number in them is a real result.

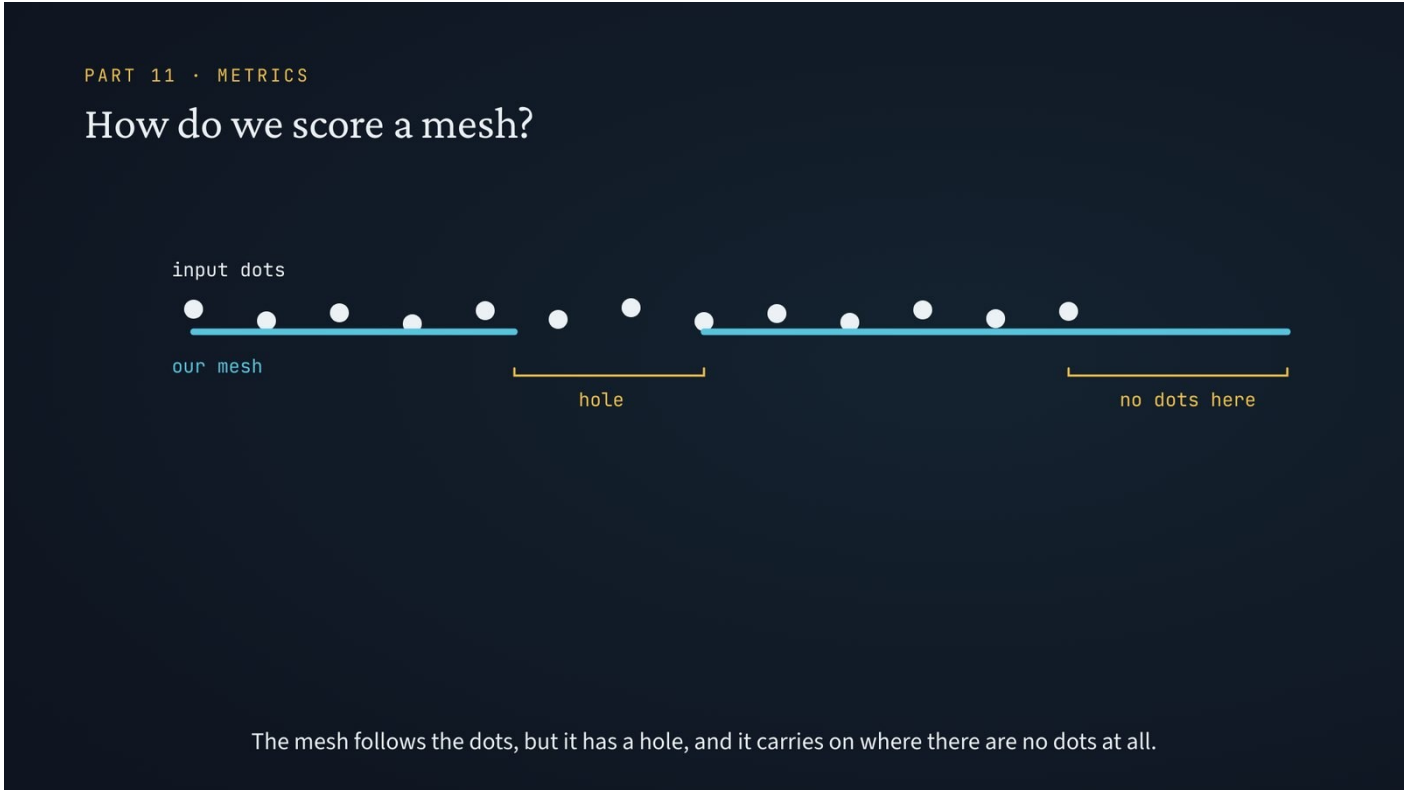
They explain each method step by step, which is why the slides before them skip formulas and methodology.

Video 1: Screened Poisson



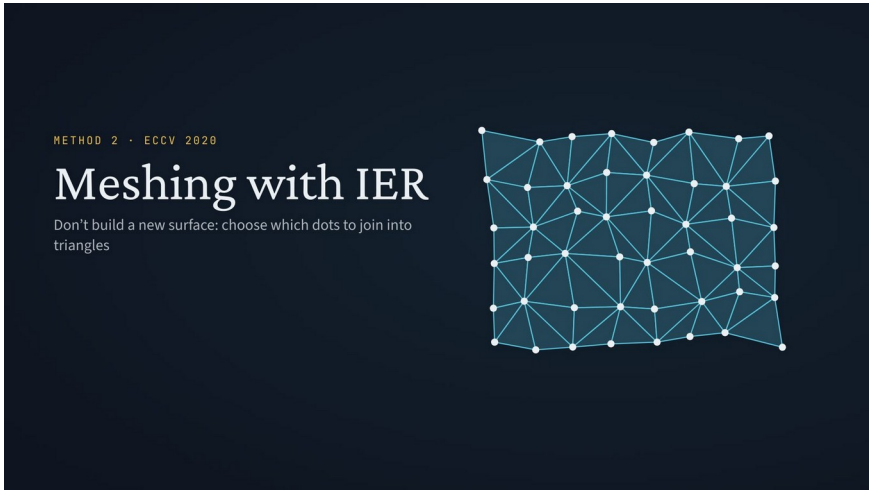
3 min 9 s · 12 scenes

Video 2: The metrics



3 min 48 s · 14 scenes

Video 3: IER meshing



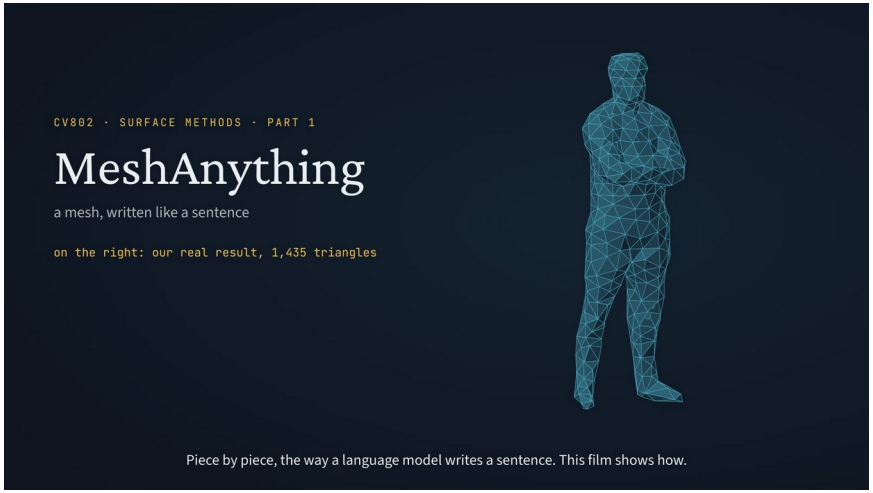
Part 1 · 2 min 23 s · 8 scenes



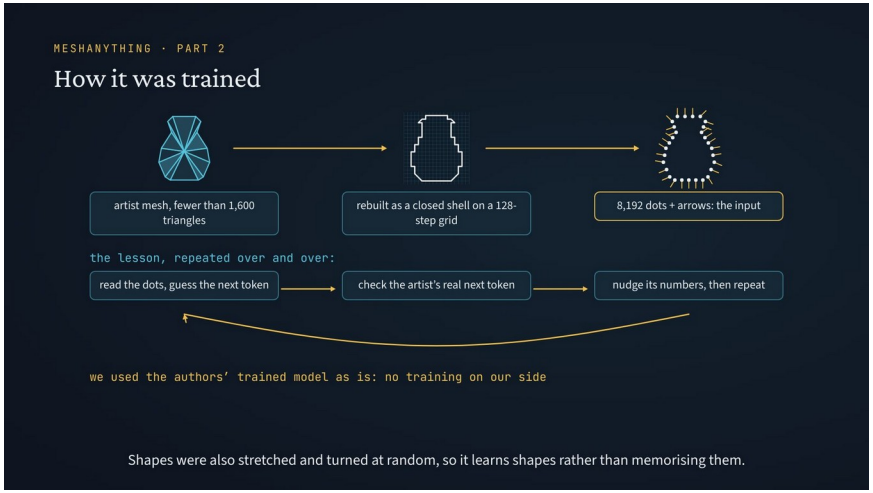
Part 2 · 2 min 28 s · 7 scenes

4 min 51 s in total

Video 4: MeshAnything



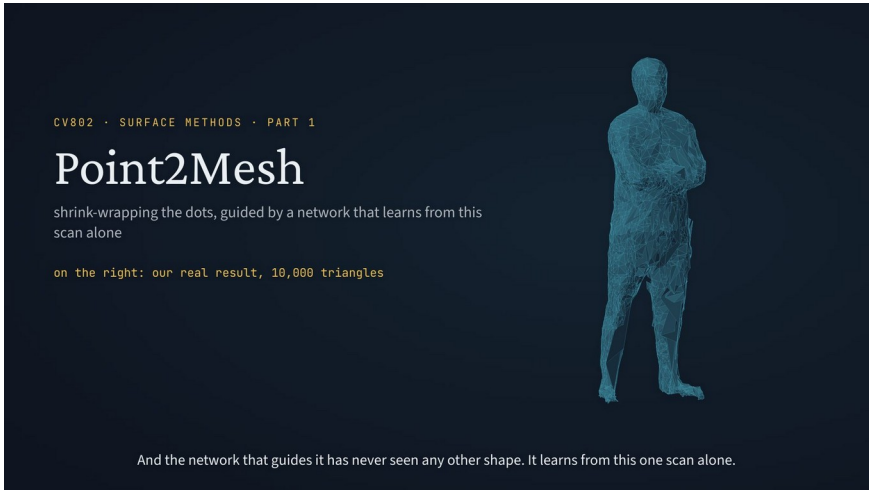
Part 1 · 3 min 22 s · 10 scenes



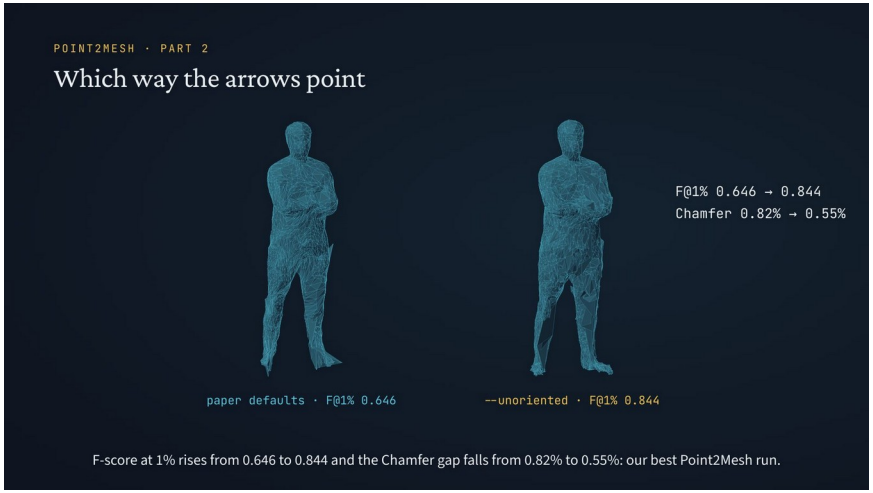
Part 2 · 3 min 2 s · 8 scenes

6 min 24 s in total

Video 5: Point2Mesh



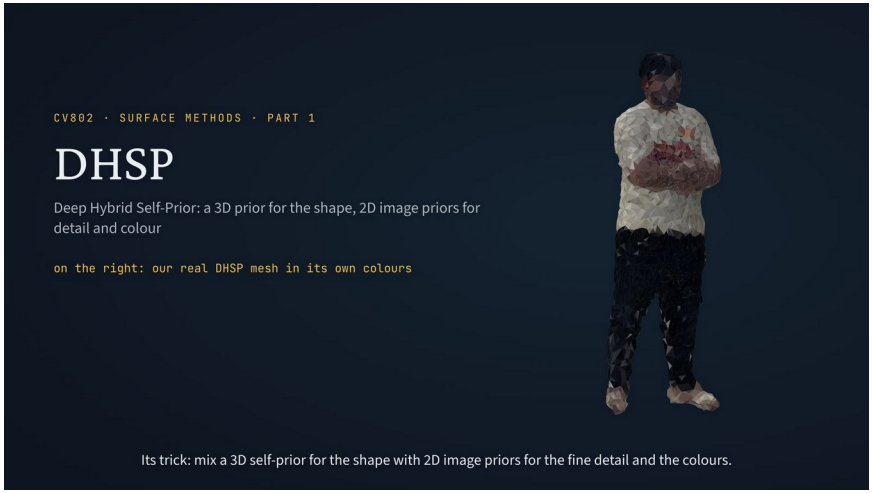
Part 1 · 3 min 8 s · 9 scenes



Part 2 · 2 min 26 s · 7 scenes

5 min 34 s in total

Video 6: Deep Hybrid Self-Prior (DHSP)



Part 1 · 2 min 52 s · 8 scenes



Part 2 · 2 min 54 s · 8 scenes

5 min 46 s in total

Video 7: NKSR

CV802 · EXTRA METHOD 1

NKSR

Neural Kernel Surface Reconstruction: Poisson's idea, with building blocks a network has learned

on the right: our real NKSR mesh in its own colours



It is our fastest method: a full mesh of our scan in about 7 seconds. On the right, its result in its own colours.

4 min 6 s · 12 scenes

Extras: stills from the videos

One still per scene, in order, from the seven videos.

CV802 · SUB-PROJECT 2

Screened Poisson reconstruction

From loose dots to a closed skin, in ten steps



STEP 1

The problem

```
mesh = triangles glued edge to edge
```



Our goal: a connected skin of triangles (a mesh), saved as .obj.

STEP 2

Inside or outside?

$\chi = 1$ inside, 0 outside

χ (chi) is the indicator function

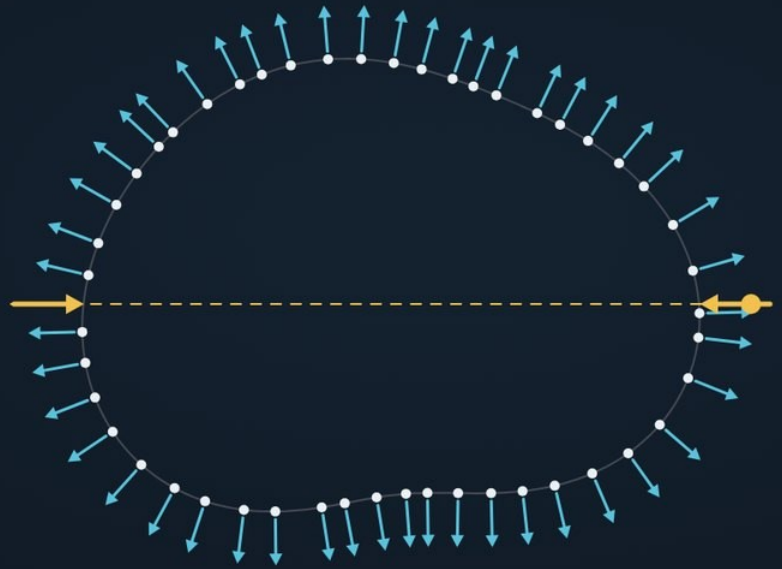


The skin is simply the border between the 1s and the 0s.

STEP 3

Arrows point outside

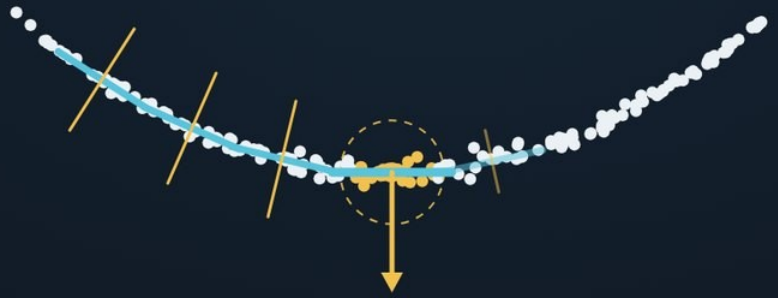




χ is flat, then jumps exactly at the skin, inward, opposite the outward arrows. So the arrows tell us how χ changes.

STEP 4

One small tile per dot

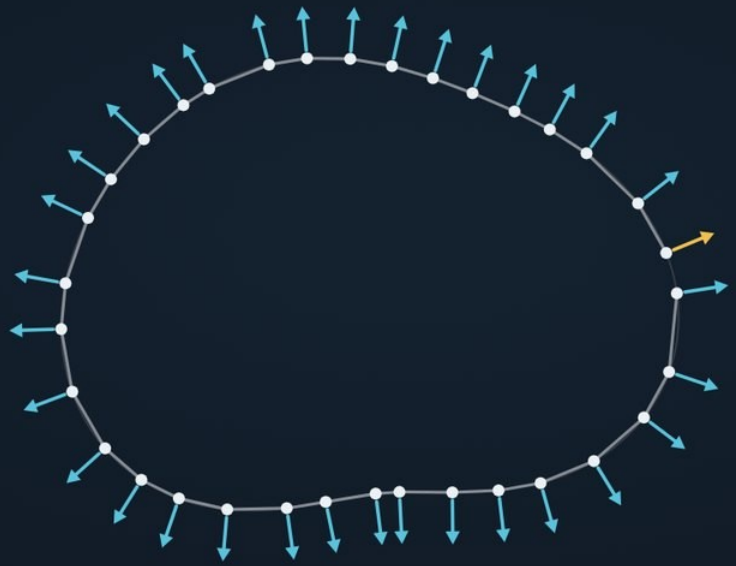


Repeat for every dot: each one gets its own tile and arrow line. Not one plane, 139,000 tiny ones.

STEP 5

Make the arrows agree

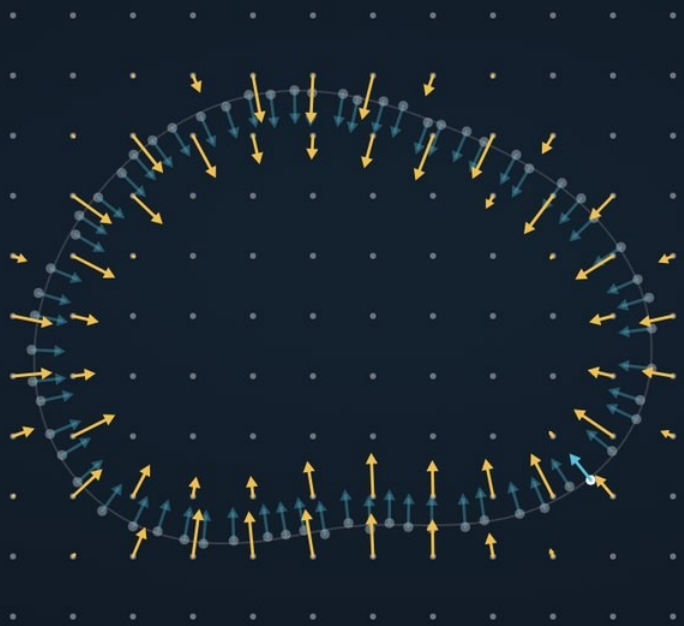
Vote: most arrows point toward the local centre, so flip them all.



Now they all agree, but they all point inward. A vote checks, and flips every arrow out.

STEP 6

Arrows onto the grid



Do this for every dot: the grid now holds an arrow field V, non-zero only near the skin.

STEP 7

Rebuild χ from its changes

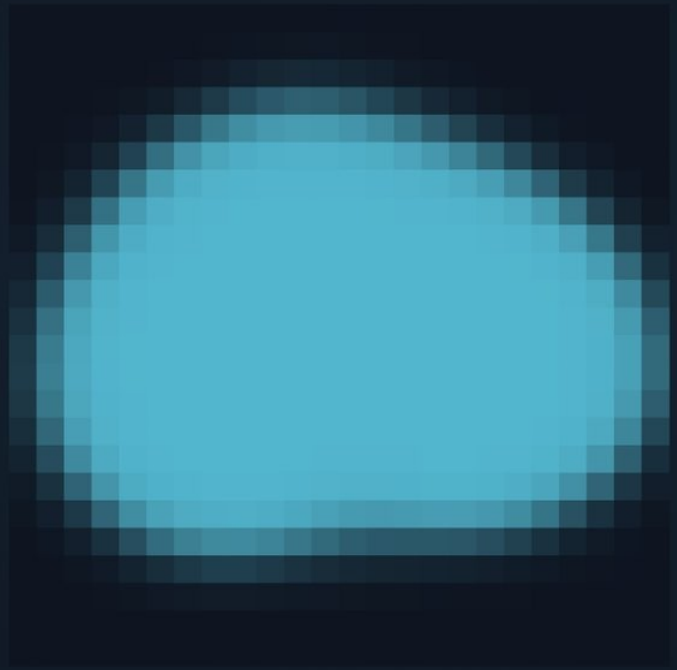
$$\Delta\chi = \nabla \cdot \mathbf{v}$$

find the χ that best agrees with every arrow

grid 192^3

CG iteration 276

error



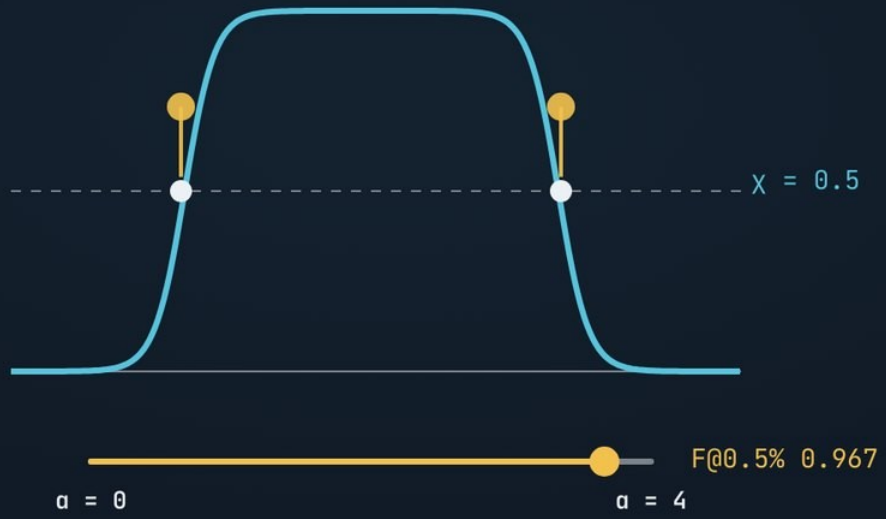
To be quick it solves a small blurry grid first, then refines it: $96^3 \rightarrow 192^3 \rightarrow 384^3$.

STEP 8

Pins keep the skin on the dots

```
wanted 0 0 0 1 1 1 0 0
step 7 0 .1 .3 .7 .9 .8 .3 .1
```

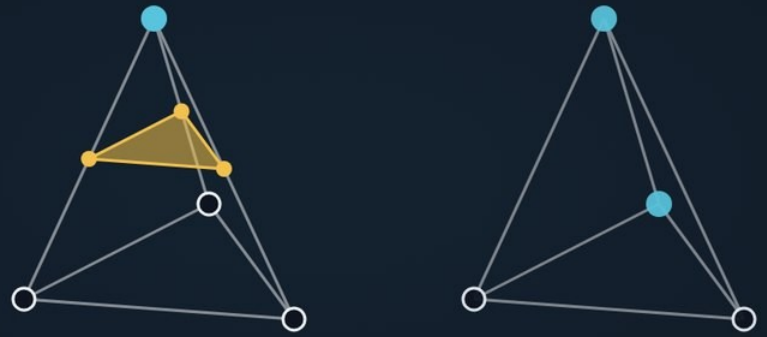
Extra rule: at every dot, χ = 0.5. The dots are pins.



α is the pin strength. 0 = classic Poisson, smooth but drifting. 4 = ours: F@0.5% 0.931 → 0.970.

STEP 9

Draw the skin



1 vs 3 → 1 triangle

2 blue and 2 white: 4 crossing points make a 4-sided patch, split into 2 triangles.

STEP 10

Trim invented skin

Trimmed: no invented skin, small openings. Best accuracy, F@0.5% 0.988.

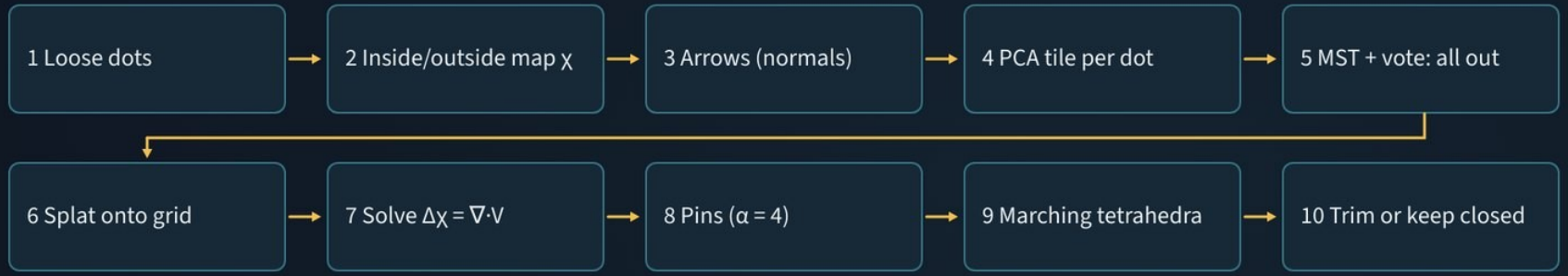
Watertight (untrimmed): closed like a balloon, needed for 3D printing. F@0.5% 0.977.



We keep both versions: trimmed for accuracy, watertight for printing.

RECAP

The whole pipeline

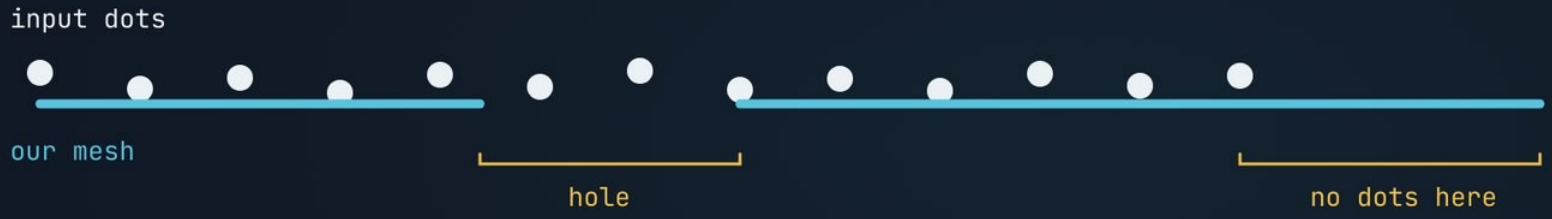


1.02 million triangles in 39 s

VGGSfM input · F@1% 0.999 · Chamfer 0.149% of the diagonal · most accurate of all 8 methods

PART 11 · METRICS

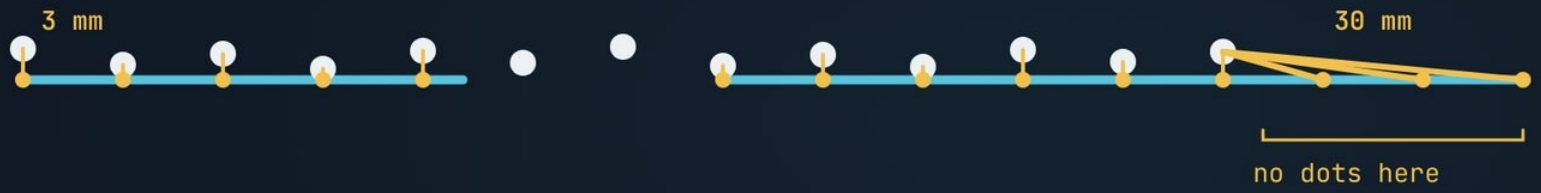
How do we score a mesh?



The mesh follows the dots, but it has a hole, and it carries on where there are no dots at all.

PART 11 · METRICS

Direction 1: mesh → dots

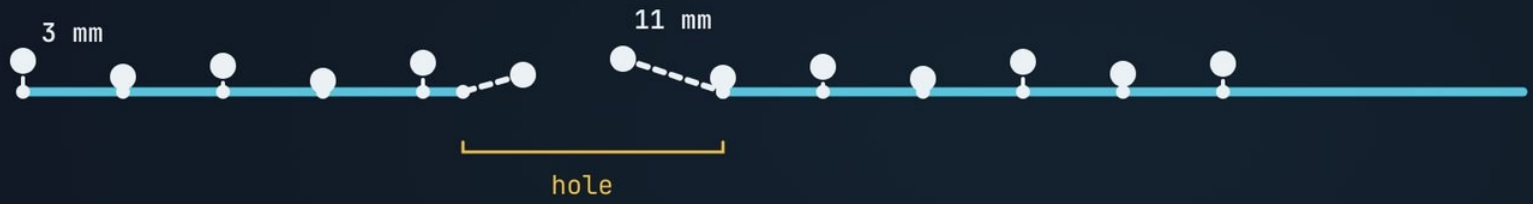


accuracy: distance from each mesh point to its nearest dot

`no dots behind the mesh → long lines = invented skin`

Where the mesh has no dots behind it, the lines get long. Accuracy catches invented skin.

Direction 2: dots → mesh



completeness: distance from each dot to the closest point on the mesh

`no mesh under the dots → long lines = a hole`

Dots above the hole have long lines: the mesh failed to cover them. Completeness catches holes.

PART 11 · METRICS

Why measure both ways?



a tiny mesh · accuracy good · completeness bad



a mesh much longer than the dots · completeness good · accuracy bad

Each one passes one test and fails the other. Only a mesh that is good both ways really fits the dots.

Chamfer: the average gap

list 1: mesh → dots (accuracy)



dashed line = average = 6.1 mm

list 2: dots → mesh (completeness)



dashed line = average = 3.1 mm

Chamfer = ½ × (mean of list 1 + mean of list 2)

= ½ × (6.1 mm + 3.1 mm) = 4.6 mm

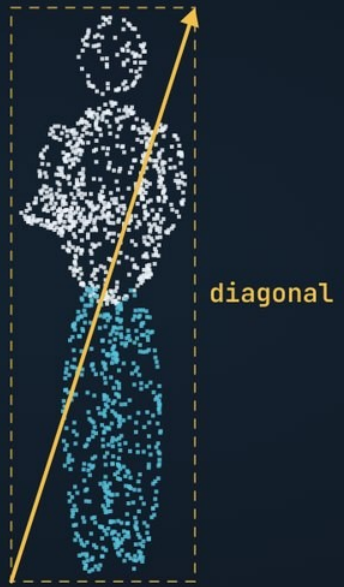
lower is better; 0 would mean the mesh passes through every dot

Lower is better: 0 would mean the mesh passes exactly through every dot.

PART 11 · METRICS

Why “% of the diagonal”?

```
diagonal = 6.01 units
Chamfer = 0.0090 units
Chamfer ÷ diagonal = 0.149 %
```



Chamfer % = (Chamfer ÷ diagonal) × 100

```
0.149 % × 1.9 m ≈ 2.8 mm (average gap)
```

Our best Poisson: Chamfer 0.149% of the diagonal. The box diagonal is about 1.9 m, so that is about 2.8 mm.

F-score: close enough?



τ = the largest distance that still counts as “close enough”

```
blue: shorter than τ → close enough
yellow: longer than τ → too far
```

Every line shorter than τ turns blue. Every longer line stays yellow: too far.

Precision, recall, F-score



$$P = 11 \div 14 = 0.79$$

$$R = 11 \div 13 = 0.85$$

$$F = 2 \times 0.79 \times 0.85 \div (0.79 + 0.85) = 0.81$$

The F-score combines the two: $F = 2PR \div (P + R)$. It is a number from 0 to 1, and higher is better.

Why not a plain average?

	precision P	recall R	plain average	F-score
a good mesh	0.99	0.99	0.99	0.99
misses half the dots	1.00	0.50	0.75	0.67
mostly invented	0.20	1.00	0.60	0.33

$F = 2PR \div (P + R)$ is pulled toward the weaker of P and R

A mostly invented mesh averages 0.60, but scores just 0.33. To score well, a mesh must be good both ways.

PART 11 · METRICS

Our Poisson numbers

Poisson run (VGGSfM)	Chamfer %	F@1%	F@0.5%
classic α=0, 256^3	0.230	0.990	0.931
screened α=4, 256^3	0.179	0.998	0.970
screened α=4, 384^3 (best)	0.149	0.999	0.988
watertight, 384^3	0.154	0.998	0.977

τ = 1% of the diagonal (≈ 2 cm) · strict τ = 0.5% (≈ 1 cm)

Chamfer = how far, on average. F-score = what share is close enough.

We report both numbers for every method: Chamfer for the average gap, F-score for the share that is close enough.

PART 11 · METRICS

Smooth or crumpled?



smooth mesh: the arrows agree → high normal consistency



crumpled mesh: the arrows point every which way → low

DHSP: before our fix 0.65 (crumpled), after our fix 0.89 (smooth)

This caught DHSP's first mesh: almost perfect F-score, but crumpled. Our fix made it smooth.

PART 11 · METRICS

Can it be printed?



closed: holds water → printable

a hole: water leaks out

printable: Poisson (untrimmed) · Shape As Points · DHSP (after our fix) · Point2Mesh

not printable: Poisson (trimmed) · IER · MeshAnything · NKSR · 2DGS

Watertight meshes can go to the printer. Open ones need repair first, or a method that closes the surface.

PART 11 · METRICS

All the scores at a glance

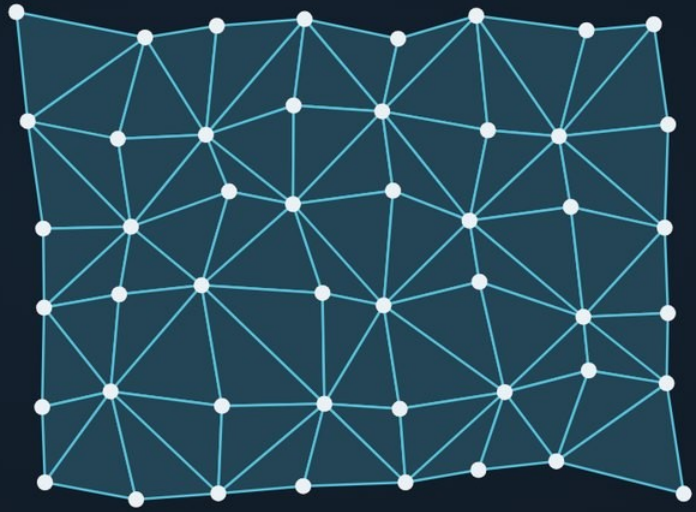
score	the question it answers	what it catches
Chamfer %	how far apart, on average?	overall closeness
F-score	what share is close enough?	invented skin, holes
Normal consistency	do the directions agree?	crumpling
Colour PSNR	do the colours match the dots?	colour errors
Watertight	is the mesh closed?	not printable

No single number is enough, so for every method we report all of them, and we also look at the shaded pictures.

METHOD 2 · ECCV 2020

Meshing with IER

Don’t build a new surface: choose which dots to join into triangles



IER · PART 1

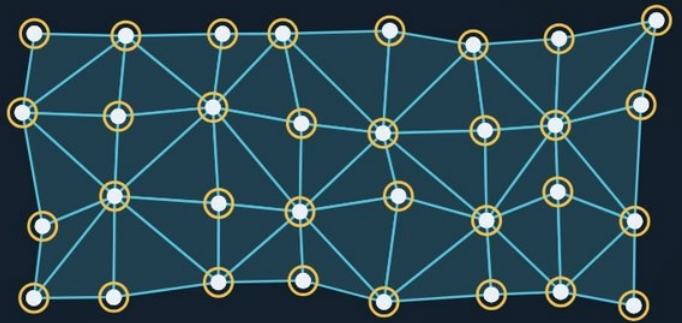
A different idea

Poisson: makes a new surface



corners are new points (yellow)

IER: joins our own dots



corners are our dots (rings)

So the whole question becomes: which triangles are real? IER answers it by choosing, not by solving.

IER · PART 1 · STEP 1

Propose candidate triangles



8 neighbours → 28 candidate triangles

50 neighbours (what IER uses) → 1,225 per dot

12,500 dots → millions of candidates, most of them wrong

Only a few candidates belong to the real skin (blue). The rest of IER is about picking exactly those.

The trap: close is not connected



So “nearest neighbours” happily builds triangles across the gap (yellow). Distance alone cannot tell real from fake.

IER · PART 1

Two kinds of distance

ruler (straight): 56

ant (walking): 2,057

walking ÷ straight ≈ 36

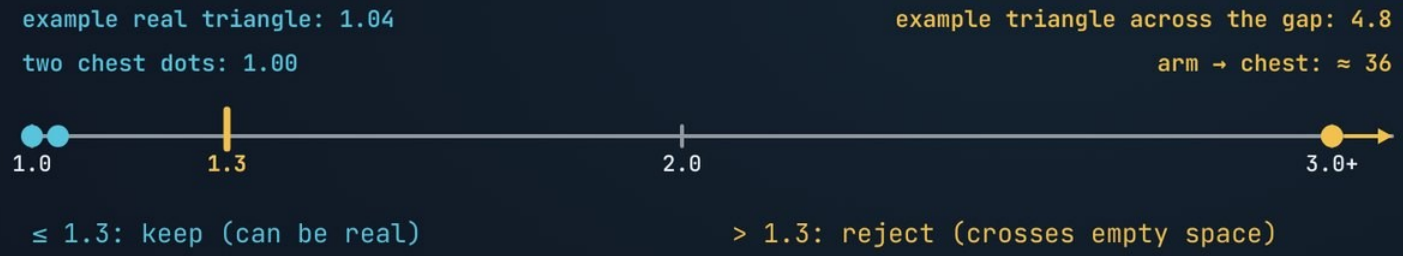


Arm to chest: the ruler is short, but the ant must go up the arm, over the shoulder and back down. A huge ratio.

The intrinsic–extrinsic ratio

IER = walking distance ÷ straight distance

for a triangle: its 3 walks added ÷ its 3 straight edges added



IER uses a cut-off of 1.3. Up to 1.3 the triangle can be real; above it, it crosses empty space.

Three kinds of candidates

good



ratio ≤ 1.3, on the surface

okay



ratio ≤ 1.3, cuts off the surface

bad



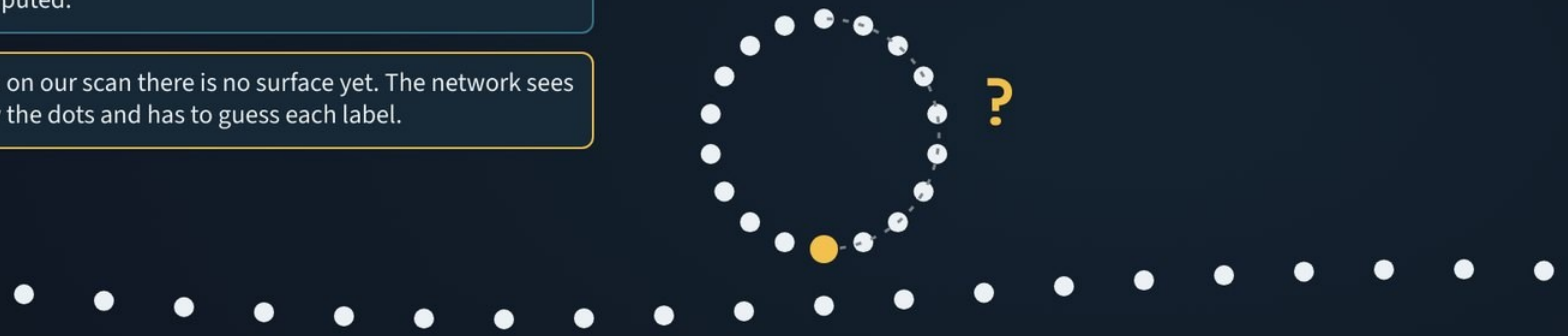
ratio > 1.3, crosses a gap

Bad: ratio above 1.3. The triangle jumps across empty space. Bad candidates are never used.

The catch

Training: on about 23,000 shapes whose true surface is known, the real ratio and label of every candidate are computed.

Use: on our scan there is no surface yet. The network sees only the dots and has to guess each label.



Next, Part 2: the network, and building the mesh

Part 2 shows how that network looks at the dots, and how the chosen triangles are joined into one mesh.

IER · PART 2

How IER was trained

A known shape: one of about 23,000 ShapeNet objects → Sample about 12,500 dots on its surface → Propose candidates from 50 neighbours, as in Part 1 → Measure the true ratio by walking on the real surface → Label every candidate: good, okay or bad



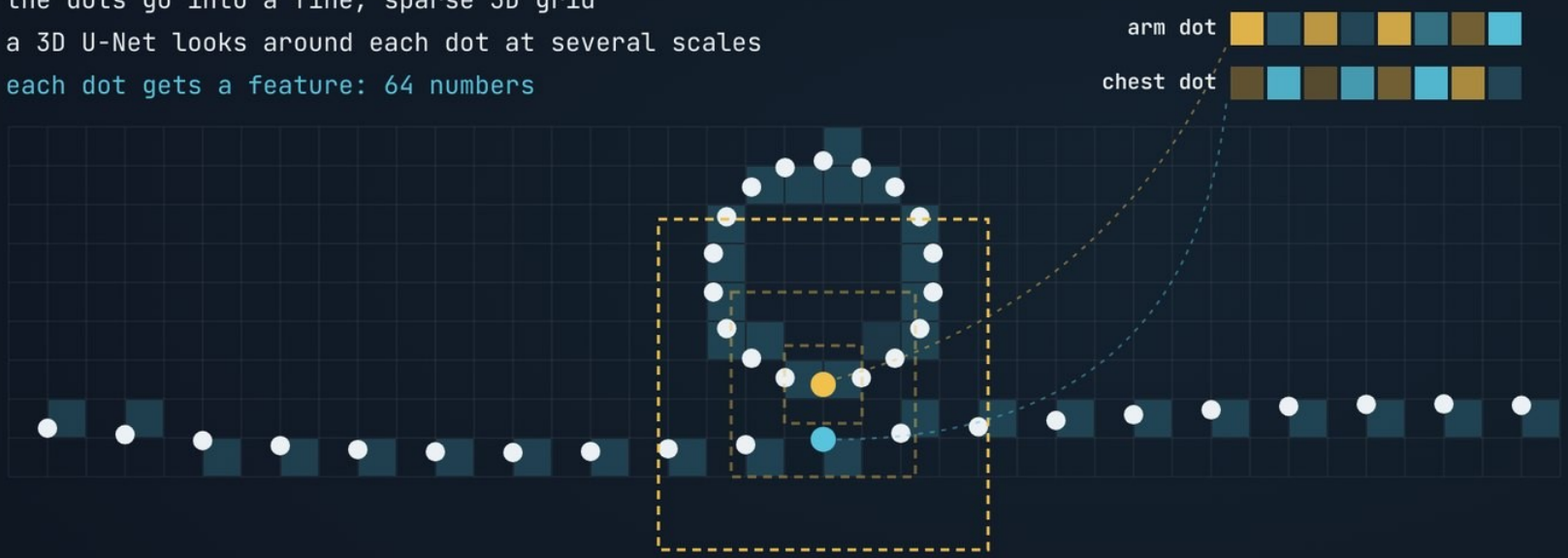
then a network learns: dots in → label out

A network is then trained to predict those labels from the dots alone. Once trained, it needs no surface.

IER · PART 2

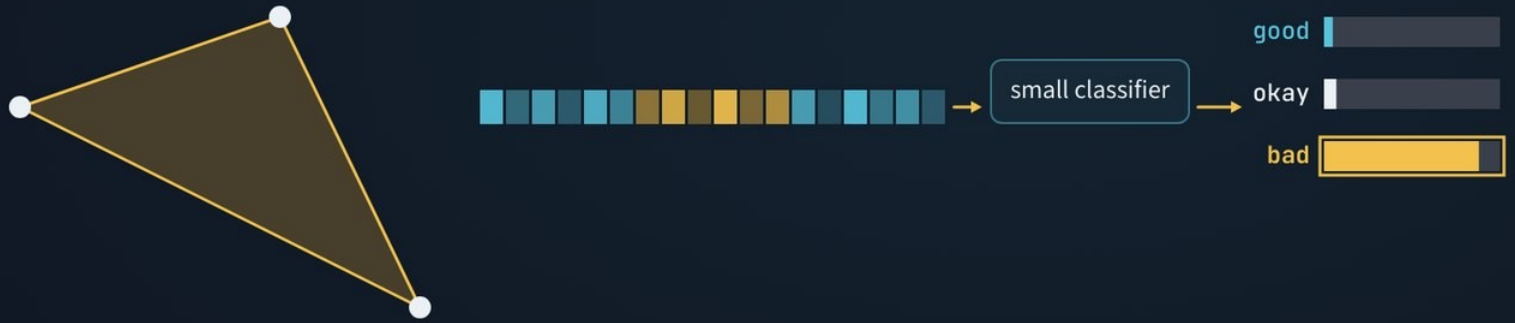
What the network sees

1. the dots go into a fine, sparse 3D grid
2. a 3D U-Net looks around each dot at several scales
3. each dot gets a feature: 64 numbers



A dot on the arm and the dot just below it on the chest get different features, even though they are close in space.

Judging one triangle

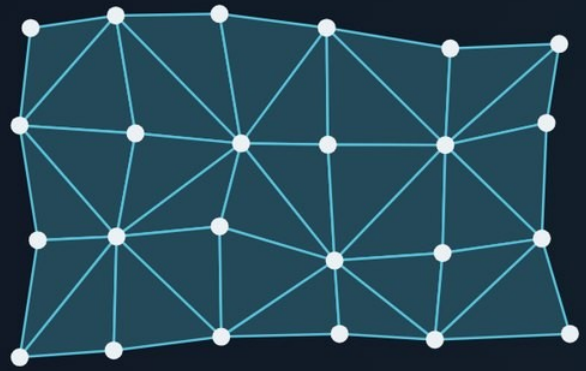


second example: a triangle across a gap

A triangle across a gap gets a high “bad” score. Millions of candidates are judged at once on the GPU: about 30 s for our scan.

IER · PART 2

Building the mesh



1. Throw away every candidate labelled bad.

2. Sort the rest: good before okay, small triangles first.

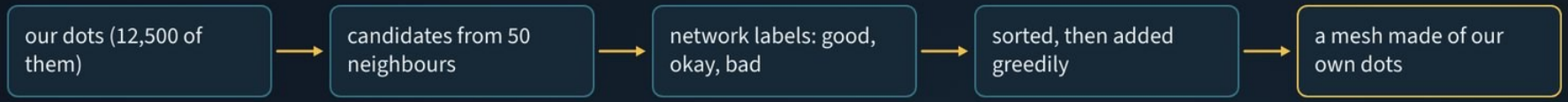
3. Add a triangle only if it crosses no triangle already in the mesh, and none of its edges would get a third triangle.

```
added: 30          rejected: 15
```

What survives is the mesh: every corner is one of our dots, and every triangle passed the network and the checks.

IER · RECAP

IER in one line



Poisson: solves for a brand-new surface

IER: chooses triangles between our own dots

`next method on the roadmap: MeshAnything`

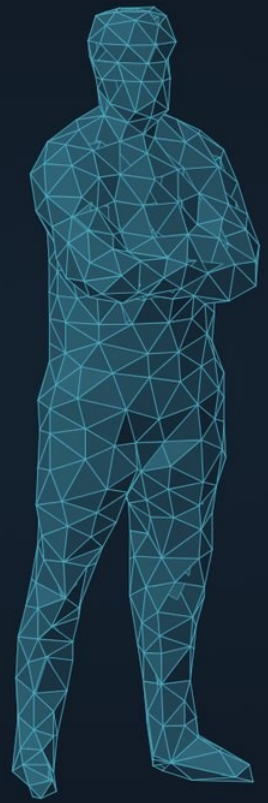
Poisson solves for a new surface. IER chooses among triangles of our own dots. Two very different answers to the same question.

CV802 · SURFACE METHODS · PART 1

MeshAnything

a mesh, written like a sentence

on the right: our real result, 1,435 triangles



Piece by piece, the way a language model writes a sentence. This film shows how.

MESHANYTHING · THE GOAL

Two kinds of mesh

like a scanner



191 pieces

like a 3D artist



15 pieces

That tidy kind is called an artist mesh, the kind people build by hand for games. MeshAnything is trained to make it.

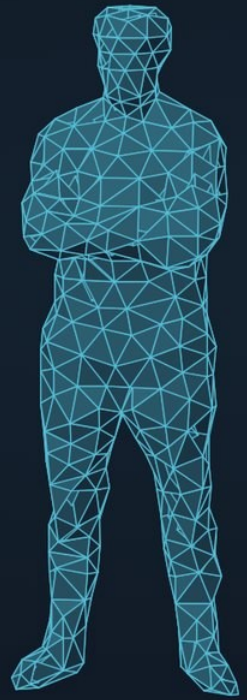
MESHANYTHING · OUR SCAN

The same person, two meshes



1,023,553

triangles · Poisson (ours)



1,435

triangles · MeshAnything

about 700 times fewer

About 700 times fewer triangles for the same person. Light enough for a game, though fine detail is lost.

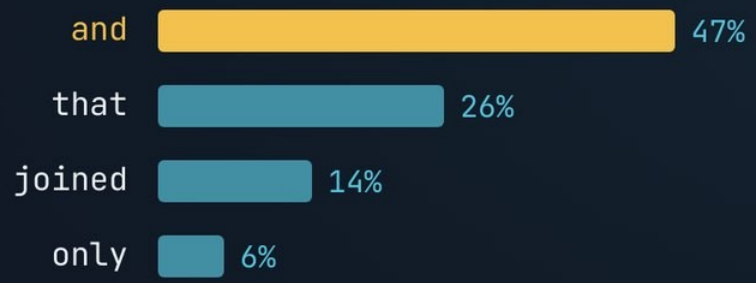
MESHANYTHING · THE IDEA IT BORROWS

How a language model writes

the sentence so far

A | triangle | has | three | corners | and

its score for every possible next token



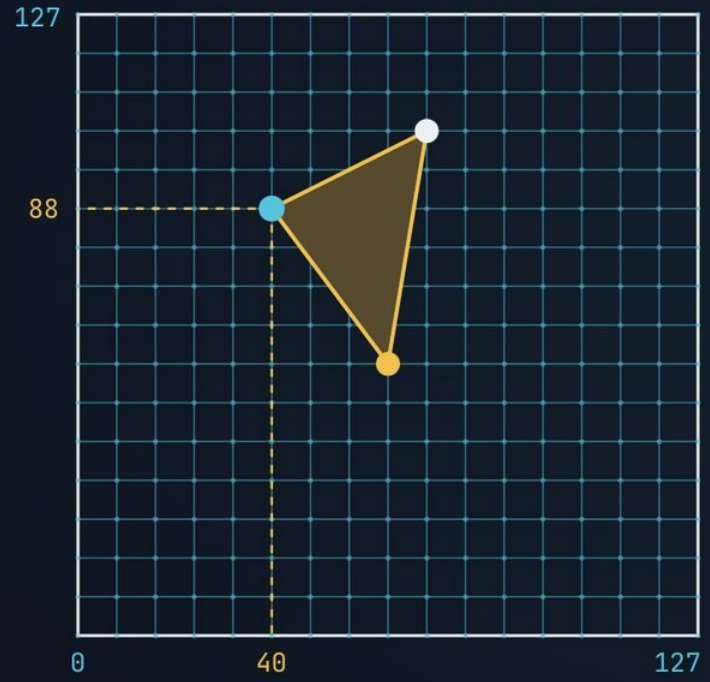
greedy: always take the top score

sampling: a weighted roll of the dice

Always taking the top score is called greedy. Picking at random, weighted by the scores, is called sampling.

MESHANYTHING · TOKENS

Corners into numbers



each side cut into 128 equal steps (16 drawn here)

corner → (40, 88, 64)

x, y read off the grid; z (depth) is cut the same way

triangle → 9 numbers

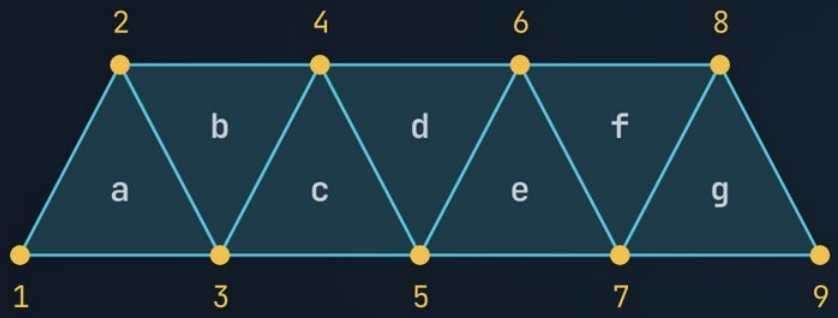
40 88 64 72 104 56 64 56 80

a mesh → one long list of numbers

dictionary: 128 numbers + 4 marks = 132 tokens

Its whole dictionary is only 132 tokens: the 128 numbers plus a few marks such as start, end and new strip.

A fixed reading order



listed in a random order:

d b g a f c e

f a c g b e d

b e a d g c f

one mesh, three different sentences

real rule: by z, then y, then x

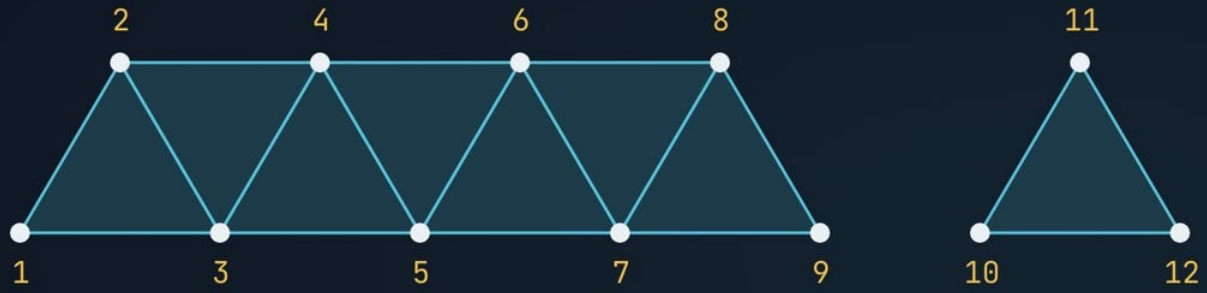
(1 2 3) (2 3 4) (3 4 5) (4 5 6) (5 6 7) (6 7 8) (7 8 9)

one mesh → always the same sentence

Each triangle is then written by its corner numbers, smallest first. Now one mesh always gives one sentence.

MESHANYTHING V2 · ITS KEY TRICK

Share the edge



plain: every triangle in full (each chip is one corner = 3 numbers) — 72 tokens

1 2 3 2 3 4 3 4 5 4 5 6 5 6 7 6 7 8 7 8 9 10 11 12

V2: reuse the last edge, add only the new corner — 37 tokens

1 2 3 4 5 6 7 8 9 & 10 11 12

about half the tokens, same mesh

our mesh: 1,435 triangles · 12,915 numbers plain · about 6,100 with sharing

About half the tokens for the same mesh. Shorter sentences let V2 write bigger meshes: about 1,600 triangles, double V1.

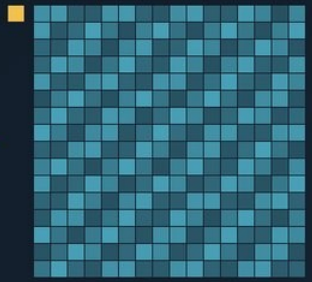
MESHANYTHING · THE MODEL

Where the shape comes in



8,192 dots + their arrows

shape encoder (pretrained)



257 shape tokens

1 summary + 256 details

written first, like a prompt

transformer

… 40 88 64 72 104 56 64 56 80

each new token looks back at the prompt and at everything already written

It was trained so that, given a shape, the right next token gets the top score. Part 2 shows that training.

MESHANYTHING · OUR REAL RESULT

Watch it write



triangles written

1,435

tokens written

6,117

strips (each new one after a &)

259

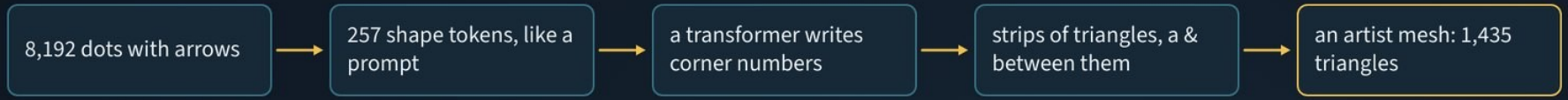
greedy: always the top score

END

Then it writes the END token and stops: 1,435 triangles in about 45 seconds on our GPU.

MESHANYTHING · RECAP

Part 1 in one line



Poisson: solves for a surface

IER: chooses triangles between our dots

MeshAnything: writes a new mesh, token by token

`Part 2: training, why our raw dots failed, our experiments`

Three methods, three ideas: solve, choose, write. Part 2 shows how it was trained and what happened on our scan.

MESHANYTHING · PART 2

How it was trained



artist mesh, fewer than 1,600 triangles



rebuilt as a closed shell on a 128-step grid



8,192 dots + arrows: the input

the lesson, repeated over and over:

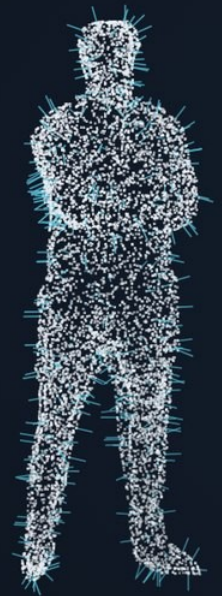
read the dots, guess the next token → check the artist's real next token → nudge its numbers, then repeat

we used the authors' trained model as is: no training on our side

Shapes were also stretched and turned at random, so it learns shapes rather than memorising them.

MESHANYTHING · ON OUR SCAN

The input matters



clean dots, as in training

even · smooth · closed



our raw scan dots

138,677 dots · uneven · gaps



V2 fed the raw dots

F@1% 0.51 · Chamfer 1.27%

Like reading messy handwriting after only ever seeing print. The model is fine; the input is unfamiliar.

MESHANYTHING · ON OUR SCAN

The fix

our raw dots (138,677) → our Poisson mesh, closed (Video 1) → their marching-cubes step, 128 steps → 8,192 clean dots + arrows → MeshAnything V2

before: raw dots



after: through our mesh



F@1%: 0.51 → 0.987

Chamfer: 1.27% → 0.30%

triangles: 1,802 → 1,435

also: turned so up is +Y, fitted into the bo

then moved back onto our scan afterwards

F-score at 1% jumps from 0.51 to 0.99 and the Chamfer gap shrinks four times. Same model; only the input changed.

Our experiments

input	model	grid	decoding	triangles	Chamfer	F@1%
raw dots	V1	–	greedy	795	1.40%	0.44
raw dots	V2	–	greedy	624	4.58%	0.11
raw dots	V2	–	sampling	1,802	1.27%	0.51
dense mesh	V1	128	greedy	471	0.42%	0.93
dense mesh	V2	128	greedy	1,435	0.30%	0.987
dense mesh	V2	256	greedy	1,810	0.31%	0.972
dense mesh	V2	128	sampling	2,058	0.34%	0.96

grid = steps of the marching-cubes rebuild · sampling: two seeds, both about 0.96

Chamfer is the average gap as a % of the box diagonal; F@1% is the share of points within 1%. See the Metrics video.

MESHANYTHING · EXPERIMENTS

What we learned

1. The input matters most: raw dots 0.51, through our mesh 0.987
2. V2 beats V1: more triangles allowed, more shape kept
3. Greedy beats sampling: the dice add stray triangles
4. Keep the grid it was trained on: 128 steps is enough

128-step grid
1,435 triangles · F 0.987 · 77 s

256-step grid
1,810 triangles · F 0.972 · 105 s

Lesson four: keep the grid it was trained on. 128 steps did as well as 256, and faster.

MESHANYTHING · HONEST VIEW

Limits and strengths

- needs a clean dense mesh first: on the noisier MVS cloud, F@1% fell to 0.87
- about 1,500 triangles: no room for a face, fingers or wrinkles
- corners snap to 128 steps: on a 1.8 m person, about 1.4 cm each
- a few holes and flipped triangles: not watertight, so not printable as is
- one token at a time: about 45 seconds for some 6,000 tokens
- strength: a light, tidy, artist-style mesh, ready for games and phones



Its strength: a light, tidy mesh like an artist would build, ready for games, phones and quick previews.

MESHANYTHING · RECAP

In one line

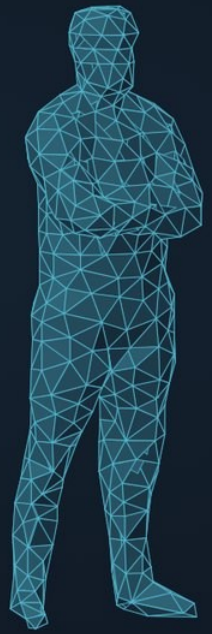
Poisson: solves for a surface

IER: chooses triangles between our dots

MeshAnything: writes a new mesh, token by token

shape → 257 tokens → a transformer writes corners → a light artist mesh. On a real scan, feed it a clean dense mesh first.

`next on the roadmap: Point2Mesh`



On a real scan, give it a clean dense mesh first. Next on the roadmap: Point2Mesh.

CV802 · SURFACE METHODS · PART 1

Point2Mesh

shrink-wrapping the dots, guided by a network that learns from this scan alone

on the right: our real result, 10,000 triangles



And the network that guides it has never seen any other shape. It learns from this one scan alone.

POINT2MESH · THE IDEA

Shrink-wrap

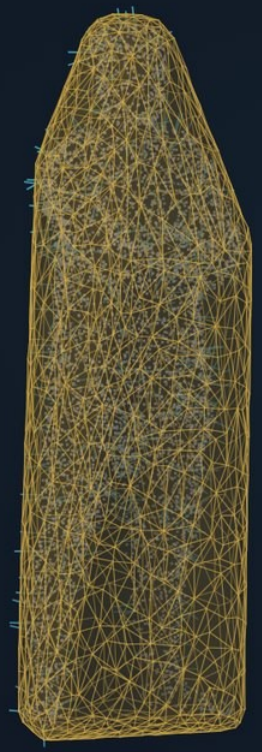
1 · start loose: the convex hull
2 · pull it tight onto the dots
3 · one closed skin, no holes



The result is one closed skin with no holes. That is Point2Mesh in a picture; the rest is how it pulls.

POINT2MESH · ON OUR SCAN

The starting skin



30,000 dots

each with its arrow from Video 1

2,000 triangles

the convex hull, rebuilt by Manifold

closed: no holes, from start to finish

A tool called Manifold rebuilds it as a clean closed mesh of 2,000 triangles. This is the skin we start from.

Pushing the corners

push: a small step for every corner

goal: shrink the gap to the dots

problem: the noise gets copied



gap to the dots

(the Chamfer distance)

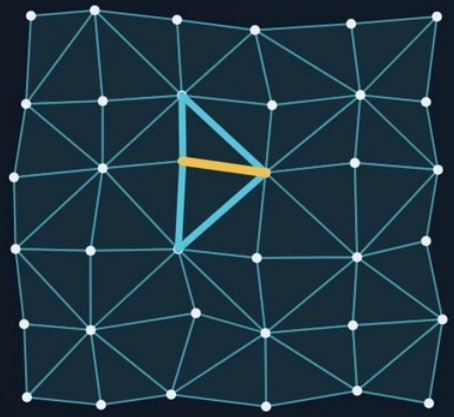
dashed: the real, smooth edge

A scan always has noise. We want a skin that follows the shape, not every wobble. That needs something smarter.

POINT2MESH · THE TRICK

Let a network push

input: 6 fixed random numbers per edge



one small filter, the same at every edge

mesh CNN (+ 3 residual blocks)



each corner: the average push of all its edges

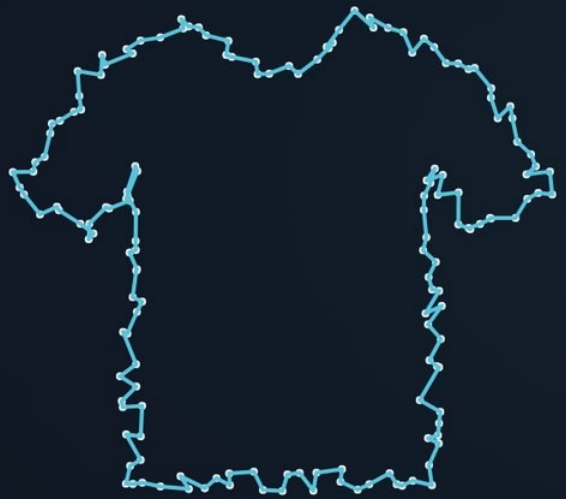


gap to the dots → update the weights, repeat

No dataset and no pretraining: the network starts random and is trained on this one scan alone.

The self-prior

corners pushed freely



copies every wobble

illustration

pushed by the network



shape first, noise much later

So the shape is fitted long before the noise. This built-in taste for clean shapes is called a self-prior.

POINT2MESH · THE GOAL

What it minimises

$$\text{loss} = \text{gap} + 0.1 \times \text{arrows} + 0.1 \times \text{evenness}$$



1 · gap to the dots

15k → 25k points on the skin

vs our 30,000 dots



2 · arrows agree (× 0.1)

triangle arrow vs dot arrow

--unoriented: line, not sign



3 · even triangles (× 0.1)

neighbours: similar areas

no slivers, no giants

POINT2MESH · OUR REAL RUN

Coarse to fine



step

10,000

triangles

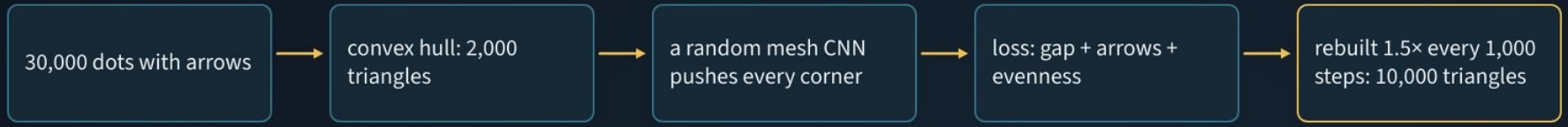
10,000

at 10,000: split into 2 parts

10,000 steps and about 42 minutes later: our Point2Mesh result. Closed, smooth, 10,000 triangles.

POINT2MESH · RECAP

Part 1 in one line



Poisson solves, IER chooses, MeshAnything writes,

Point2Mesh shrink-wraps, with a network trained on this scan alone

Part 2: why the arrows' direction mattered, stray dots, our runs

Its one shared filter prefers clean shapes, so it fits the body before the noise. Part 2: what happened on our scan.

POINT2MESH · PART 2

Which way the arrows point



paper defaults · F@1% 0.646



--unoriented · F@1% 0.844

F@1% 0.646 → 0.844

Chamfer 0.82% → 0.55%

F-score at 1% rises from 0.646 to 0.844 and the Chamfer gap falls from 0.82% to 0.55%: our best Point2Mesh run.

POINT2MESH · EXPERIMENTS

Our four runs

run	steps	max tri.	arrows	F@1%	Chamfer	time
first try	6,000	16,000	default	0.602	0.85%	21 min
paper settings	10,000	10,000	default	0.646	0.82%	44 min
paper + unoriented	10,000	10,000	unoriented	0.844	0.55%	42 min
bigger and longer	15,000	20,000	unoriented	0.655	0.80%	3 h 22 min

the arrows' direction mattered most: 0.646 → 0.844

more triangles and more steps: no better, five times slower

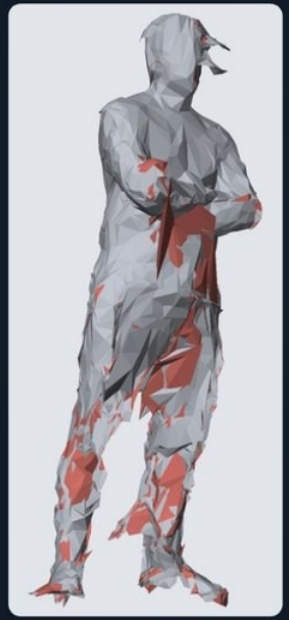
Twice the triangles and 50% more steps did not help: 0.655, and five times slower.

POINT2MESH · OTHER INPUT

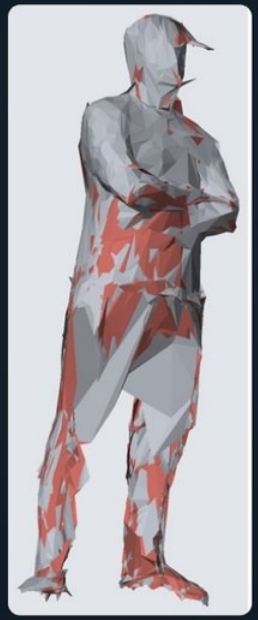
Stray dots



illustration



raw MVS
F@1% 0.133



MVS-clean
F@1% 0.387

the MVS cloud has stray dots floating in the air

a shrink-wrap must cover them all, so it stretches out membranes

404 strays removed: 0.133 → 0.387
still far below VGGSfM: 0.844

Removing 404 stray dots helps, to 0.387, but folds remain and it stays far below the VGGSfM result of 0.844.

The red patches

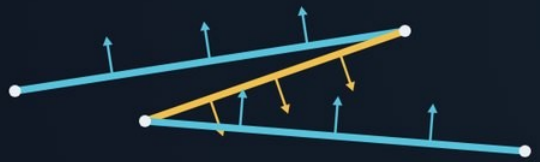
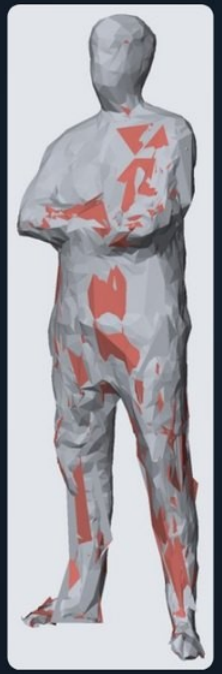


illustration: a skin folding over



red = the inside of the skin

the skin folded over itself there

still closed: no holes

unoriented arrows cannot tell inside from outside

A likely side effect of the unoriented arrows: to that term, inside and outside look exactly the same.

POINT2MESH · RESULTS

Against the others

method	triangles	Chamfer	F@1%	NC	closed	time
Poisson (ours)	1,023,553	0.149%	0.999	0.75	no	39 s
IER	17,488	0.187%	0.998	0.82	no	29 s
MeshAnything	1,435	0.295%	0.987	0.82	no	77 s
Point2Mesh	10,000	0.553%	0.844	0.80	yes	42 min



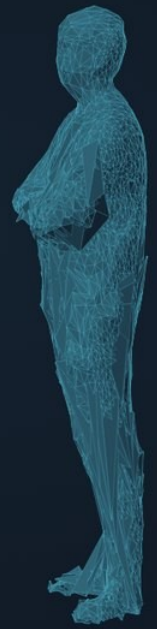
the least accurate of the four on our scan,
but the only closed mesh, and no training data

The price: about 42 minutes on our GPU, against well under two minutes for each of the other three.

POINT2MESH · HONEST VIEW

Limits and strengths

- slow: about 42 minutes on our GPU, where the others take seconds
- it must cover every dot: stray dots pull out membranes
- needs good arrows: flipped ones held the default run back
- folds where the skin turns inside out: the red patches
- detail is capped by 10,000 triangles; more did not help
- strengths: always closed, smooth, no training data, ready to 3D print



Its strengths: the mesh is always closed and smooth, it needs no training data, and it is ready to 3D print.

POINT2MESH · RECAP

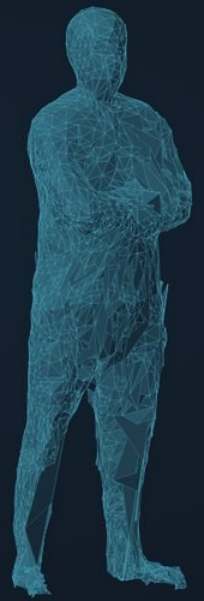
In one line

Poisson solves, IER chooses, MeshAnything writes,

Point2Mesh shrink-wraps

hull → a fresh, untrained network pulls it onto the dots → rebuilt finer every 1,000 steps. On a real scan: unoriented arrows and a clean cloud.

`next on the roadmap: DHSP, which builds on Point2Mesh`



On our scan, unoriented arrows and a clean cloud mattered most. Next: DHSP, which builds on Point2Mesh.

CV802 · SURFACE METHODS · PART 1

DHSP

Deep Hybrid Self-Prior: a 3D prior for the shape, 2D image priors for detail and colour

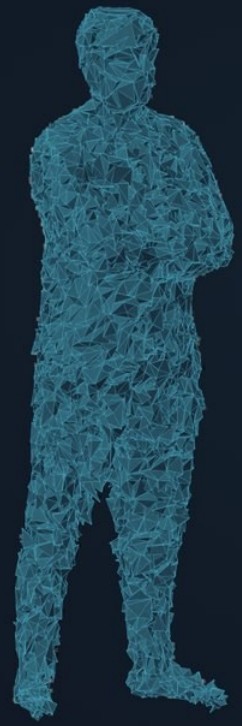
on the right: our real DHSP mesh in its own colours



Its trick: mix a 3D self-prior for the shape with 2D image priors for the fine detail and the colours.

DHSP · STAGE 1

A 3D self-prior first



triangles

16,000

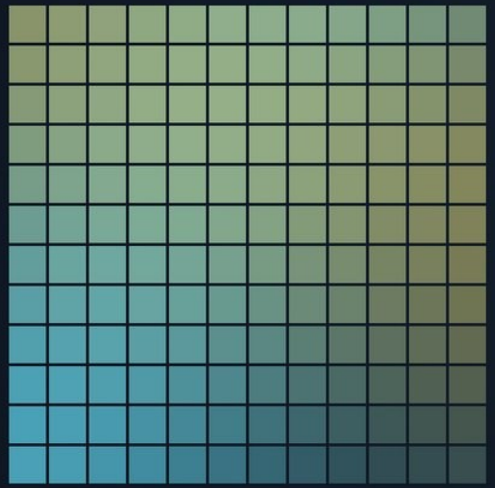
after round 4

loss: gap + 0.2 × edge length
4 rounds, 2,000 → 3,456 steps each
at 16,000: worked on in 4 parts

From afar the result looks good. Up close it hides a problem, and Part 2 is about it.

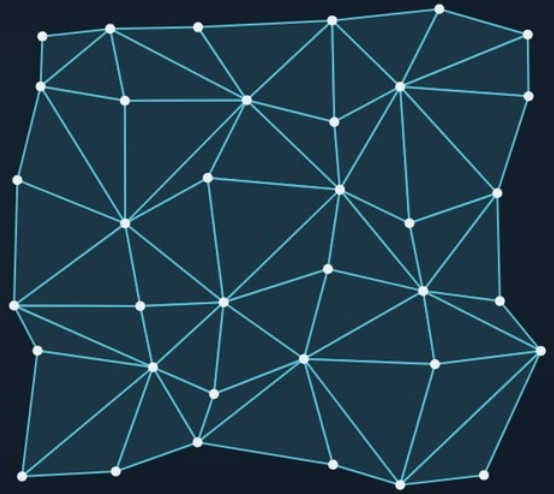
DHSP · THE 2D IDEA

Why pictures



a picture: a neat grid

an image prior
fills the gap



a mesh: no rows, no columns

so DHSP first turns the mesh surface into a picture

A mesh has no rows or columns. So DHSP turns the surface into a picture first, by flattening it.

DHSP · UV MAP

Flatten the skin



This is our real mesh. With this map, anything painted in the square can be wrapped straight back onto the body.

DHSP · 2D GEOMETRY PRIOR

A picture of the shape

red: left ↔ right
green: down ↔ up
blue: back ↔ front



shown on our final mesh

Reading positions back moves the mesh corners to where the picture says. Then the 3D prior polishes once more, for 1,200 steps.

DHSP · 2D TEXTURE PRIOR

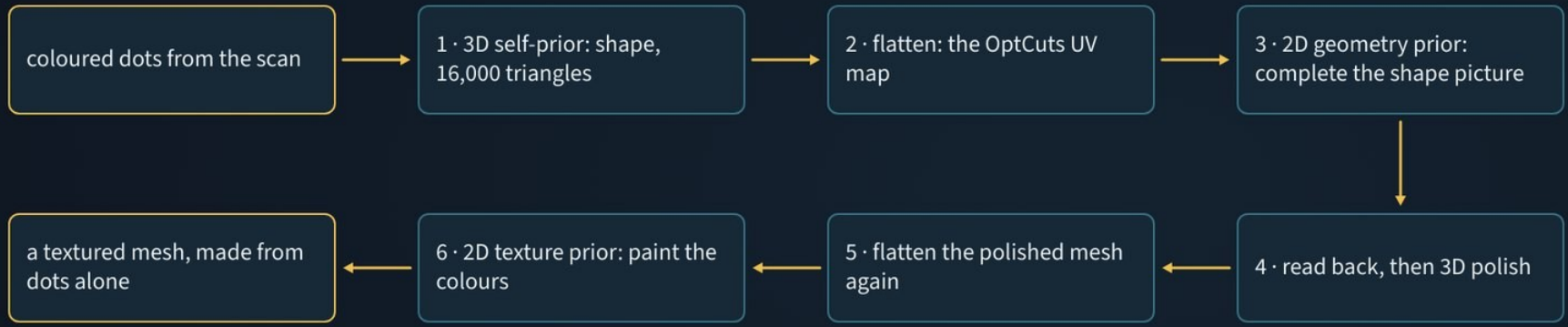
Painting the colours



Wrapped back through the UV map, the texture colours the mesh: DHSP’s own colour, made from the dots alone.

DHSP · RECIPE

The whole pipeline



a 3D prior for the overall shape, 2D priors for detail and colour

Read back and polished in 3D, flattened again, and a second 2D prior paints the texture. Every network trains on this scan alone.

DHSP · RECAP

Part 1 in one line

Point2Mesh: one 3D self-prior

DHSP: a 3D self-prior plus two 2D image priors

shape from the 3D prior; the mesh flattened into a UV square; a picture of the shape and a picture of the colours, both completed by image priors

`Part 2: our scan, the crumpled prior, and our fix`



Part 2: what happened on our scan, why the full recipe would not run at first, and how we fixed it.

DHSP · PART 2

The crumpled prior



F@1% 0.999

as close to the dots as Poisson

NC 0.65

but the surface is crumpled

42% fold

of neighbouring triangle pairs

after round 2: only 1%

The dots are matched, but the surface between them is folded. And that breaks the next step of the recipe.

DHSP · ON OUR SCAN

The recipe stalls

a crumpled skin, seen edge-on



laid flat, the folds overlap



OptCuts:
stopped at 15 h

the fallback, xatlas: many small pieces



every edge of a piece is a seam

A faster tool, xatlas, cuts the skin into many small pieces instead.
But every cut is a seam, and the 2D priors smear across seams.

DHSP · OUR CONTRIBUTION

Our fix: rebuild first



3D prior: 16,000 triangles, 42% fold

Manifold rebuild + simplify: 3,000 triangles, then 6,000

OptCuts: 15 h unfinished
→ done in seconds



after our fix: 6,000, under 2% fold

The price: a coarser mesh, 6,000 triangles instead of 16,000, so it sits a little further from the dots.

DHSP · EXPERIMENTS

Our runs

run	triangles	F@1%	NC	closed	own colour
3D prior only	16,000	0.999	0.65	yes	–
+ xatlas pieces, texture	16,000	0.999	0.65	no	15.3 dB
rebuild + OptCuts: full recipe	6,000	0.874	0.89	yes	18.2 dB
+ Laplacian smoothing	6,000	0.901	0.89	yes	18.5 dB

own colour: PSNR of DHSP's texture against the dot colours, higher is better

Laplacian: pulls every corner towards the average of its neighbours (weight 1)

A Laplacian smoothness term, pulling each corner towards its neighbours, lifts F-score at 1% to 0.901: our best DHSP run.

DHSP · OTHER INPUT

On the MVS cloud



illustration: arms touching the body make loops

F@1%

	3D prior	full recipe
MVS	0.980	0.900
MVS-clean	0.982	0.916

stray dots → thin fins in the shape picture

arms touching the body → loops, no one-piece cut

final atlas by xatlas: the mesh is not closed

And there the arms touch the body, making loops like a mug with two handles. OptCuts cannot cut that into one flat piece, so xatlas steps in.

DHSP · RESULTS

Against the others

method	triangles	Chamfer	F@1%	NC	closed	time
Poisson (ours)	1,023,553	0.149%	0.999	0.75	no	39 s
IER	17,488	0.187%	0.998	0.82	no	29 s
MeshAnything	1,435	0.295%	0.987	0.82	no	77 s
Point2Mesh	10,000	0.553%	0.844	0.80	yes	42 min
DHSP	6,000	0.583%	0.901	0.89	yes	> 1.5 h

the smoothest surface so far, closed, with its own colour,
but by far the slowest



Its 3D prior alone matched Poisson for accuracy; the full recipe trades a little accuracy for a clean, coloured mesh.

DHSP · RECAP

In one line

a 3D prior for the shape,

2D image priors for detail and colour

On our scan the 3D prior came out crumpled, and the recipe stalled. Rebuilding it before flattening let the whole recipe run: smooth, closed, coloured.

`next: three extra methods, NKSR, Shape As Points, 2DGS`



That completes the five methods of the assignment. Next on the roadmap: the three extra methods we added.

CV802 · EXTRA METHOD 1

NKSR

Neural Kernel Surface Reconstruction: Poisson’s idea, with building blocks a network has learned

on the right: our real NKSR mesh in its own colours



It is our fastest method: a full mesh of our scan in about 7 seconds. On the right, its result in its own colours.

NKSR · THE IDEA

What it keeps from Poisson



Poisson: a full grid around the shape

both:

inside / outside

one linear system

surface = the zero line



NKSR: only near the dots

…while NKSR only builds the function near the dots, from building blocks a network has learned. Here is how, in 2D.

NKSR · TOY IN 2D

A field of bumps



cells only near the dots: sparse

each cell holds one bump that fades to zero

ask: 0 on each dot, +ε just outside, -ε just inside

$$(A^TA + \lambda I)\,\alpha = A^Tb$$

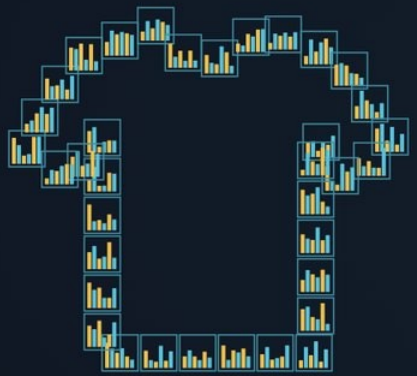
170 bumps → 170 weights, solved together

the weighted bumps add up to the function

its zero line is the surface

Add up the weighted bumps and you get the function: blue outside, gold inside. Its zero line is the surface.

Learned bumps



each cell: a list of numbers (features)

$$K(x, y) = \langle \varphi(x), \varphi(y) \rangle \cdot B(x - y)$$

bump = feature similarity × a small smooth hill



fixed bumps (toy) learned bumps

illustration

pretrained once: the “kitchen-sink” model, objects + rooms + streets

The network was trained once on a mix of objects, rooms and street scenes. It also predicts a clean arrow for each cell. We only run it.

NKSR · THE MASK

Only near the dots



here the scan has no dots

NKSR predicts how far each place is from the dots

and cuts the surface where it is more than 2 cells away

so the surface stays open at the gap

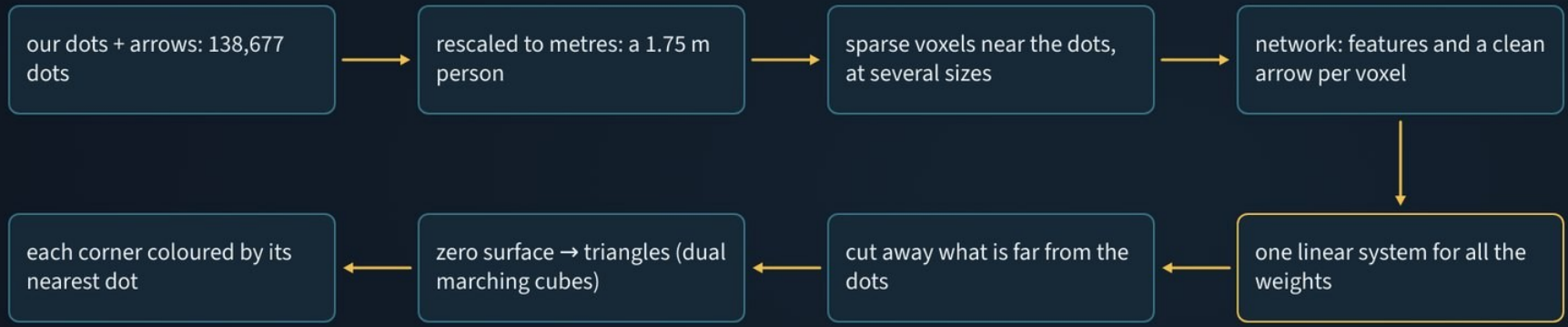
+ honest: nothing invented where there is no data

- never closed: cannot be 3D printed as it is

That is honest, but it means an NKSR mesh is never closed, so it cannot be 3D printed without repair.

NKSR · PIPELINE

The whole pipeline



our whole scan: about 7 seconds on one GPU

Then the far-away parts are cut, the zero surface is turned into triangles, and each corner takes the colour of its nearest dot.

NKSR · OUR SCAN

Our runs

run	detail	passes	triangles	time	Chamfer	F@1%	NC
full detail, 2 passes	1.0	2	8,403,162	24.5 s	0.157%	0.997	0.75
full detail, 1 pass	1.0	1	2,088,273	6.6 s	0.156%	0.998	0.75
half detail, 1 pass	0.5	1	112,286	1.4 s	0.206%	0.993	0.84

passes: extra rounds that refine the triangles where the surface is

a second pass: 4× the triangles, no gain

half detail: 18× fewer triangles, smoother, slightly less close

Half detail gives 18 times fewer triangles and a smoother surface, for a slightly larger gap. We kept full detail, one pass.

NKSR · OUR SCAN

Detail level



full detail · NC 0.75



half detail · NC 0.84

full detail: follows every dot, so scan noise becomes bumps

half detail: each bump covers more dots and averages the noise

closer, or smoother

A clear trade-off: closer to the dots, or smoother. The scores favour full detail; the eye may prefer half.

NKSR · MVS CLOUD

Best scores, holes too



Chamfer 0.080%

F@1% 1.000

the best scores of any method, on any input

yet: holes, loose flakes, and red patches
(red = the inside, seen through a hole)

it sits right on the dots, and that is all the scores measure;
where the cloud has gaps, a hole costs nothing

good scores are not a good mesh: look at it

Both are true. It sits right on the dots, which is all the scores measure; where the cloud has gaps, holes cost nothing.

NKSR · RESULTS

Against the others

method	triangles	Chamfer	F@1%	NC	closed	time	MVS Chamfer
Poisson (ours)	1,023,553	0.149%	0.999	0.75	no	39 s	0.127%
IER	17,488	0.187%	0.998	0.82	no	29 s	0.120%
MeshAnything	1,435	0.295%	0.987	0.82	no	77 s	0.518%
Point2Mesh	10,000	0.553%	0.844	0.80	yes	42 min	3.977%
DHSP	6,000	0.583%	0.901	0.89	yes	> 1.5 h	0.636%
NKSR	2,088,273	0.156%	0.998	0.75	no	6.6 s	0.080%

as close as Poisson, the fastest, best on MVS,
but huge, bumpy on noisy dots, and never closed

A strong, fast choice for accurate open surfaces; not one for 3D printing.

NKSR · RECAP

In one line

Poisson's recipe, with bumps a network has learned,

built only near the dots

pretrained, done in seconds, millions of triangles; as close as Poisson and best on MVS; but open, and it trusts every dot, noise included

next: Shape As Points, a Poisson solver that can be trained through

Next on the roadmap: Shape As Points, which turns the Poisson step itself into something a network can learn through.